

A systematic review on the engineering of software for ubiquitous systems



Alejandro Sánchez Guinea*, Grégory Nain, Yves Le Traon

Interdisciplinary Center for Security, Reliability and Trust, University of Luxembourg, 4 rue Alphonse Weicker, L-2721 Luxembourg, Luxembourg

ARTICLE INFO

Article history:

Received 16 October 2015

Revised 11 April 2016

Accepted 13 May 2016

Available online 16 May 2016

Keywords:

Empirical software engineering
Evidence-based software engineering
Systematic review
Research synthesis
Software development cycle
Ubiquitous systems
Development methods
Pervasive systems

ABSTRACT

Context: Software engineering for ubiquitous systems has experienced an important and rapid growth, however the vast research corpus makes it difficult to obtain valuable information from it.

Objective: To identify, evaluate, and synthesize research about the most relevant approaches addressing the different phases of the software development life cycle for ubiquitous systems.

Method: We conducted a systematic literature review of papers presenting and evaluating approaches for the different phases of the software development life cycle for ubiquitous systems. Approaches were classified according to the phase of the development cycle they addressed, identifying their main concerns and limitations.

Results: We identified 128 papers reporting 132 approaches addressing issues related to different phases of the software development cycle for ubiquitous systems. Most approaches have been aimed at addressing the implementation, evolution/maintenance, and feedback phases, while others phases such as testing need more attention from researchers.

Conclusion: We recommend to follow existing guidelines when conducting case studies to make the studies more reproducible and closer to real life cases. While some phases of the development cycle have been extensively explored, there is still room for research in other phases, toward a more agile and integrated cycle, from requirements to testing and feedback.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

Ubiquitous Computing is aimed at integrating computation with the physical world in a way that blends in so completely that it becomes unnoticeable (Want and Perring, 2005). The last 20 years or so have brought a considerable progress towards this aim with the proliferation of connected devices of different scales and ownership models (Abowd, 2012). This progress has inspired a great deal of research and development on areas such as pervasive computing, Ambient Intelligence (AmI), Internet of Things (IoT), cyber-physical systems, etc., which while under the umbrella of ubiquitous computing is focused on different specific aspects.

Back in 1999, Gregory D. Abowd brought to the attention of software engineering researchers the area of ubiquitous computing at the International Conference on Software Engineering (ICSE) (Abowd, 1999), arguing about the importance of soft-

ware engineering for the development of ubiquitous systems. From that year to date there has been a rapid growth of the field and its diversification, which has derived into a very large and generally unorganized research corpus containing approaches to address a variety of issues for different phases of the software engineering cycle for ubiquitous systems. Given the size and heterogeneity of the literature (e.g., a quick search on google scholar may provide not less than ten thousand results), and in the light that to the best of our knowledge only informal surveys restricted to specific issues have been conducted, we have seen the need to organize and synthesize the existent research corpus to obtain a clear idea on the main approaches that have been proposed for the different phases of the development of software for ubiquitous systems, what issues they addressed, what their limitations are, and what open issues still need to be tackled.

A systematic literature review is a systematic approach to building a body of knowledge about a particular topic or research question(s), and to identify problems for future research and support decision making and technological selection (Zhang and Babar, 2013). The systematic review herein presented seeks to find, evaluate, and synthesize the most relevant approaches that have been

* Corresponding author.

E-mail addresses: alejandrosanchezguinea@uni.lu (A. Sánchez Guinea), gregory.nain@uni.lu (G. Nain), yves.lettraon@uni.lu (Y. Le Traon).

proposed to date in the literature for any of the issues related to the software development life cycle for ubiquitous systems. We classify the approaches according to the phase they relate to in the development life cycle and the general concerns they address. It is our belief that this review will be useful to practitioners, willing to stay up-to-date on approaches, their limitations, and unresolved general issues, as well as for researchers trying to identify issues that have been researched or where research is lacking. The results provided by the review should be relevant for the software intensive industry.

The article is organized as described next. In [Section 2](#), we provide an overview of software development for ubiquitous system, talk about the phases that have been recognized in the literature as part of the software development life cycle of this type of systems, and identify any existing related review. [Section 3](#) describes the method that has been applied for undertaking this review. [Section 4](#) presents the results of the review, including information about the main phases considered for the development of software for ubiquitous systems, the main approaches for each phase, issues that they address and their limitations, and the issues that still remain open in general. [Section 5](#) discusses the benefits and limitations of the review, and analyses the strength of evidence and implications for both research and practice. [Section 6](#) concludes the paper.

2. Background - engineering of software for ubiquitous systems

Ubiquitous computing, as envisioned by [Weiser \(1993\)](#) more than 20 years ago, enhances computer use by making many computers available throughout the physical environment, while making them effectively invisible to the user.

In the years that have passed since Weiser vision numerous successful prototypes have been built and evaluated, demonstrating the utility of many different aspects of ubicomp systems, however many challenges are still to be solved before we can realize many of the core ubicomp scenarios on any large scale ([Caceres and Friday, 2011](#)). Due to the key role that software plays in ubiquitous systems, an important number of the challenges of ubicomp systems are related to the need for software engineering advances to support the research endeavors in ubiquitous computing ([Abowd, 1999](#)). This need has led software engineering researchers to propose, over the last 15 years or so, a large number of approaches for different phases of the engineering process specifically tailored for the characteristics of ubiquitous systems ([Abowd, 2012](#)).

2.1. Summary of previous reviews

Reviews, surveys, and state of the art overviews have been prevalent in the literature on software development for ubiquitous systems (e.g., [Shi et al., 2011](#); [Raychoudhury et al., 2013](#); [Krupitzer et al., 2015](#); [Martin et al., 2011](#)). Lots of articles have been published about the challenges of this type of development, most of the time based on rather informal literature reviews and the authors' experience (e.g., [Abowd, 2012](#); [Conti et al., 2012](#); [Tang et al., 2011](#); [Cook et al., 2009](#); [Want and Pering, 2005](#); [Edwards and Grinter, 2001](#); [Abowd, 1999](#)).

The reviews that can be found in the literature related to the development of software for ubiquitous systems have typically focused on general concepts such as autonomic computing ([Huebscher and McCann, 2008](#)) and self-adaptive systems ([De Lemos et al., 2013](#)), as well as on particular implementations of ubiquitous systems such as smart home ([Solaimani et al., 2015](#); [Alam et al., 2012](#); [Chan et al., 2008](#)) and ambient intelligence ([Cook and Das, 2007](#); [Cook et al., 2009](#)), specific phases of

the development life cycle (e.g., feedback [Brun et al., 2009](#), design [Tang et al., 2011](#)), specific tasks such as situation identification ([Ye et al., 2012](#)), specific concerns such as application mobility ([Yu et al., 2013](#)), or particular techniques or tools such as middleware ([Raychoudhury et al., 2013](#); [Martin et al., 2011](#)) and models at runtime ([Szvetits and Zdun, 2013](#)).

Among the reviews we found, only one deals with software engineering approaches but targets a more general type of systems than ubiquitous systems, that is, self-adaptive systems ([Krupitzer et al., 2015](#)). All reviews we found followed an informal process except for two, one about smart home ([Solaimani et al., 2015](#)) and one about models at runtime ([Szvetits and Zdun, 2013](#)).

2.2. Objectives of this review

The area of software development for ubiquitous systems has considerably grown in the last decade, with an important amount of approaches being proposed for addressing different concerns on each phase of the development life cycle. However, as seen in [Section 2.1](#), to the best of our knowledge no systematic review covers the most relevant phases of the ubiquitous systems' software development cycle. Therefore, this review aims at filling this gap by providing a systematic review of the literature on that subject.

To this end, this review will use the following research questions as a guide:

- RQ1 What are the stages of the development life cycle of software for ubiquitous systems that have been mainly considered?
- RQ2 What are the main approaches that have been proposed for each of these stages?
- RQ3 What are the current limitations of such approaches?
- RQ4 What are the open issues to be further investigated in regard to the development of software for ubiquitous systems?

3. Review method

In order to conduct this review, we broadly followed the guidelines by [Kitchenham and Charters \(2007\)](#) and their updated version by [Kitchenham and Brereton \(2013\)](#). We first performed a search and selection process to obtain the primary studies that constituted the basis for obtaining our results. The primary studies were passed through a quality assessment step to provide validation of the quality of each study. Next, data related to approaches' characteristics and limitations, future work, and general open issues was extracted from the primary studies. Finally, we synthesized the data to obtain the results of the review.

The workload of the review was distributed among the authors of this paper according to their experience. The two less experienced researchers (first and second authors in order of appearance on the title page, a PhD student and a post-doctoral researcher, respectively) were responsible for directly performing the review, while the most experienced researcher (third author, professor for more than nine years) was responsible for checking and verifying each of the intermediate and overall outcomes of the review, seeking to ensure consistency and completeness.

3.1. Search and selection process

The process followed to search and select primary studies is depicted in [Fig. 1](#). We have devised this process inspired by the multi-stage process used by Kitchenham and Brereton, in what can be considered the updated guidelines for conducting SLR's in software engineering ([Kitchenham and Brereton, 2013](#)).

First, an informal search step (pre11 in [Fig. 1](#)) was performed to ensure having enough papers for the review and enough

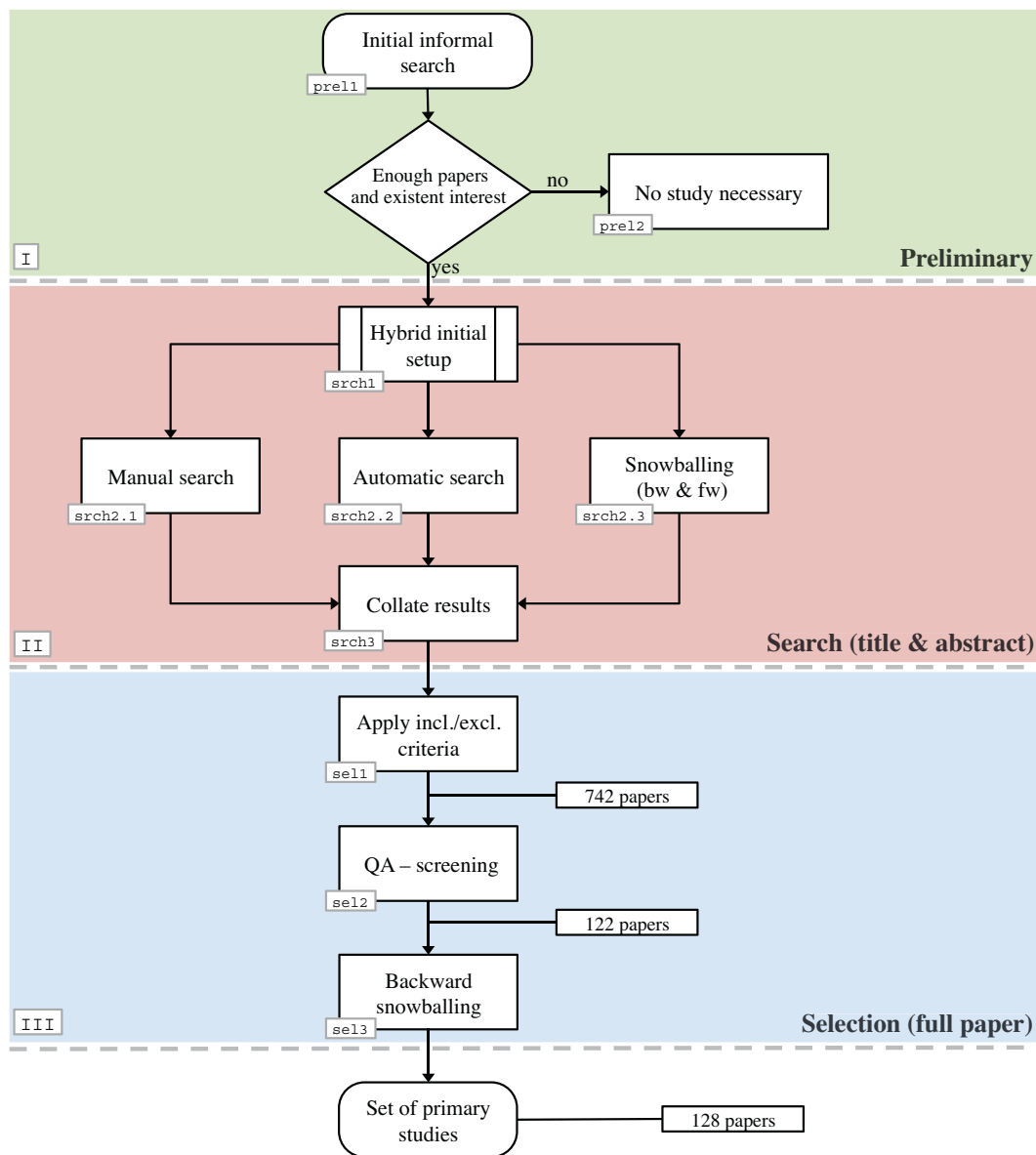


Fig. 1. Search and selection process.

interest towards the topic, as expressed by the works in the literature. Then, we performed a search process (I in Fig. 1) divided into three phases: i) ii) a hybrid initial setup (srch1 in Fig. 1) based on three different types of pilot searches; iii) a search phase divided into manual search (srch2.1 in Fig. 1), automatic search (srch2.2 in Fig. 1), and snowballing (srch2.3 in Fig. 1); and iv) a final phase (srch3 in Fig. 1) in which the outcomes from the previous phase were collated. At that point we had collected not less than 5377 papers. These papers were passed to a selection process (I in Fig. 1) where, we apply inclusion/exclusion criteria (sel1 in Fig. 1) and quality assessment (QA) screening (sel2 in Fig. 1) with which we selected 122 papers. After that, a complementary backward snowballing process (sel2 in Fig. 1) was performed to obtain a comprehensive final set. From this last step we obtained 6 more papers, making a final total of 128 papers selected as primary studies for the obtention of the results of the review.

In Section 3.1.1 we detail how the hybrid initial setup was performed. In Section 3.1.2 we explain how we conducted our main

search process. Finally, Section 3.1.3 explains how the results of the search process were analyzed and selected to obtain the set of primary studies.

3.1.1. Hybrid initial setup

To setup our main search processes (srch2.1, srch2.2, and srch2.3 in Fig. 1) we followed a hybrid approach in which we performed three pilot searches independently (one automatic, one manual, and one snowballing), and, from the combination of the outcomes of the three pilots, we determined the initial setup for each of the main search processes. The goal of our hybrid approach was to try to eliminate, right before start, the possible bias caused by our previous knowledge, which could affect the findings of the search. Thus, instead of deciding on the sources for the search based only on our previous knowledge, we let the literature complement, correct, and enrich our setups by using results from pilot search processes.

Our hybrid initial setup process is summarized in Fig. 2. As it can be observed, the process is divided into three high-level phases across the three processes of search. The phases are:

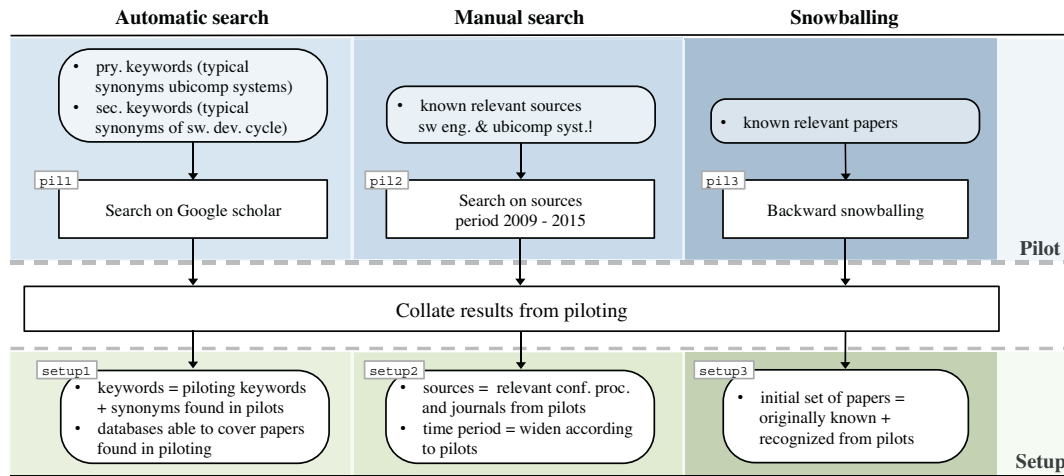


Fig. 2. Hybrid initial setup.

pilot searches, collate results from pilots, and setup of main search processes. Next we describe these three stages in detail.

Stage 1. Pilot. For each type of search process (i.e., manual, automatic, and snowballing) we performed a pilot as follows:

- Pilot of automatic search (pil1 in Fig. 2): This pilot was performed using search strings that combined any of the terms 'ubiquitous systems', 'pervasive system', or 'cyber-physical system', with any of the terms 'software development life cycle' or 'software engineering'. We considered Google Scholar as source, since it has been recognized as a good alternative to avoid bias in favor of a particular publisher (Wohlin, 2014). In this way, we benefited from a broad view of the whole spectrum of available publications.
- Pilot of manual search (pil2 in Fig. 2): This pilot was performed using as sources the proceedings of 4 conferences and 4 journals. The conference proceedings considered were Ubiquitous Computing (UbiComp), Conference on Pervasive Computing and Communications (PerCom), International Conference on Software Engineering (ICSE), and Foundations of Software Engineering (FSE). The journals used were Transactions on Software Engineering (TSE), Journal of Systems and Software (JSS), Pervasive and Mobile Computing, and Transactions on Software Engineering and Methodology (TOSEM). For these sources the time period considered was years 2010–2015. The main idea was not to cover all sources we knew relevant, but a small number that included the most relevant according to our initial knowledge. The time period was chosen to be short and recent, aiming for a fast process able to deliver relevant results for setting up our main search processes.
- Pilot of snowballing (pil3 in Fig. 2): We formed our start set of papers for this pilot based on papers we knew relevant beforehand, but only from the same relevant sources and period of time we considered for the pilot of manual search. Then, we performed backward snowballing by checking the references of the start set of papers and selecting relevant papers. The decision to spot a relevant paper was based on formal qualitative assessment of title, abstract, and structure of the paper, as well as on a minimalistic inclusion/exclusion criteria that excluded papers not based on empirical research (as seen from its abstract) and papers less than three pages long.

Stage 2. Collate results. After the pilot stage, the results were collated to support the setup of the search processes. This included removing duplicates as well as extracting and organizing for each

paper data related to publishing source (i.e., journal or conference proceedings where the paper appeared), year, and publisher (or digital library that contains the paper). Thus, the setup of the search processes was based on this information and on the selected papers from the pilot stage.

Stage 3. Setup. The setup of each of our main search processes was done based on the collated results from the three pilot searches. Thus, the setup of the main process of one type of search was optimized based on the outcomes of the pilots of the other types of search. Specifically, the initial setup for each of the search processes was optimized in the following way:

- For *automatic search setup* (setup1 in Fig. 2), based on the collated results from the pilots, we were able to target specific digital libraries aiming at covering the papers found in the pilot process. These are: ACM DL, IEEE XPLORE, Wiley Online Library, Science Direct, and Springer. We were also able to optimize the search strings aiming at covering larger amount of papers more closely related to our research goals. On the one hand, we included more terms related to ubiquitous systems such as 'home automation' and 'smart building'. On the other hand, we removed the term 'software development life cycle' after noticing it was not useful, and we kept the term 'software engineering', considering, in addition, the possibility of finding each of their elementary terms, i.e., 'software' and 'engineering', as separate terms.
- For the *manual search setup* (setup2 in Fig. 2), we benefited from the collated results of the piloting stage by broadening the time period and sources considered as to cover more papers that could be considered relevant.
- The *snowballing setup* (setup3 in Fig. 2) was done based on the guidelines by Wohlin (2014), using as tentative start set of papers being those from the collated results of the pilot stage. Then, as suggested by Wohlin's guidelines, we looked for a definite start set of papers featuring diversity and relevancy. To this end, we selected papers only from publishing sources and years that were relevant for the manual search setup, we established a maximum number of papers to be considered per publishing source, and used the number of citations of the papers to rank them and decide on which ones to keep in case too many of a single source were found.

3.1.2. Search process

The search phase of our review (II in Fig. 1) was composed of three different search processes: one based on automatic search

Table 1
Manual search sources.

Type	Abbreviation	Name	Publisher
Conference	ICSE	International Conference on Software Engineering	IEEE/ACM
	FSE	Foundations of Software Engineering	ACM
	UbiComp	Conference on Ubiquitous Computing	ACM
	PerCom	International Conference on Pervasive Computing and Communications	IEEE
	Mobiquitous	International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services	ACM
	ICCPs	International Conference on Cyber-Physical Systems	IEEE/ACM
Journal	TSE	Transactions on Software Engineering	IEEE
	TOSEM	Transactions on Software Engineering and Methodology	ACM
	–	Software: Practice and Experience	Wiley
	IST	Information and Software Technology	Elsevier
	JSS	Journal of Systems and Software	Elsevier
	–	Pervasive Computing	IEEE
	–	Pervasive and Mobile Computing	Elsevier
	–	Personal and Ubiquitous Computing	Springer
Magazine	–	Software	IEEE
	–	Computer	IEEE

on digital libraries (*srch2.1* in Fig. 1), one based on manual search on the most relevant journal and conference proceedings in the topic (*srch2.2* in Fig. 1), and one based on snowballing (*srch2.3* in Fig. 1), both backward and forward (Wohlin, 2014).

We considered articles published between 1999 and 2015. The initial date was decided based on the publication year of the first relevant article about the development of ubiquitous systems appearing in the proceedings of a top-tier conference in software engineering (specifically, The International Conference on Software Engineering, ICSE). During this stage only title and abstract were considered for each of the collected papers.

Automatic search (*srch1* in Fig. 1). This search process was performed using predefined search strings over specific electronic databases.

- Source selection: Following the updated guidelines for performing systematic reviews proposed by Kitchenham and Brereton, (2013) we selected IEEE XPORE, ACM DL, and SCOPUS as electronic databases for our automatic search. This accounts for the two major digital libraries and an indexing system (i.e., SCOPUS), which, as it has been pointed out by Chen et al. (2010) and Dyba et al. (2007), reduces the need for searching some publishers' sites such as Springer Link and Wiley Inter-science.
- Search string: Our search strings were combinations between primary search terms and secondary search terms, in both cases considering the most relevant synonyms that could be recognized. Therefore we have:

search string = *primary term* & *secondary term*

where *primary term* = {ubiquitous system, pervasive system, cyber-physical system, ambient intelligence, smart building, smart home, home automation}, and *secondary term* = {software, engineering, software engineering}.

Manual search (*srch2* in Fig. 1). This search process was conducted respecting the time period we established for all search processes (i.e., 1999–2015) over the most relevant conference proceedings and journals on ubiquitous systems and software engineering, according to our previous experience and the results obtained during the setup process. The full list of sources is presented in Table 1. The list of sources was expanded with respect to the one used during the pilot of manual search in the setup process described in Section 3.1.1.

Snowballing (*srch3* in Fig. 1). Our process of snowballing is inspired by the guidelines by Wohlin (2014). The starting set of papers necessary to begin the process was formed during the setup process as described in Section 3.1.1. The overall process is detailed in Fig. 3, and we describe next how we performed backward and forward snowballing.

The steps we followed in our *backward snowballing* process were:

1. Go through the reference list of the current paper α and exclude papers that do not fulfill the following basic criteria: publication year between 1999–2015, language of the paper English, and type of publication peer reviewed papers.
2. Remove papers that have been already examined because found earlier through either backward or forward snowballing in this or previous iteration
3. For each of the candidate papers, look for the place where it is being referenced in α to evaluate if it is relevant for our purpose based on what it is said about it.
4. For the remaining candidate papers find each paper and browse through and read its most relevant parts to evaluate its relevancy for our review (i.e., if it is about empirical research and if it is about any part of the cycle of software development for ubiquitous systems).

For our *forward snowballing* process we went through the following steps:

1. Identify candidate papers based on those citing paper α being examined (using Google Scholar).
2. From the candidate papers exclude any that does not fulfill the basic criteria used in step (1) of *backward snowballing*.
3. Same as step (2) of *backward snowballing*.
4. For each of the remaining candidate papers, look for the place in such paper where α is being cited to evaluate how relevant is the link to α and decide to keep or remove the candidate.
5. Same as step (4) in *backward snowballing*.

3.1.3. Study selection strategy

Our selection strategy consisted of three steps as shown in Fig. 1: apply inclusion/exclusion criteria (*sel1* in Fig. 1), apply quality assessment screening (*sel2* in Fig. 1) to guarantee minimum quality on the studies considered, and a complementary backward snowballing process (*sel3* in Fig. 1) to try to ensure that no relevant paper was missed. Next, we detail each of these steps.

```

input: Starting set of papers  $S_0$  built during hybrid setup
Define set  $S$  of selected papers ;
Define set  $E$  of papers for examination ;
Define set  $S_c$  of papers selected in current iteration ;
Define set  $R$  of excluded (or rejected) papers ;
 $E \leftarrow S_0$  ;
while  $E$  is not empty do
    foreach paper in  $E$  do
        /* avoid duplicates from  $S$ ,  $S_c$ , and  $R$  */
        backward snowballing and put the result in  $S_c$  ;
        forward snowballing and put the results in  $S_c$  ;
    end
     $R \leftarrow R \cup (E - S_c)$  ;
     $E \leftarrow S_c$  ;
     $S \leftarrow S \cup S_c$  ;
end
return  $S$ 

```

Fig. 3. Snowballing process.

Table 2
Quality criteria.

-
1. Is the study reported empirical research?
 2. Were the aims and objectives clearly reported (including a rationale for why the study was undertaken)?
 3. Was there an adequate description of the context in which the research was carried out?
 4. Is the presented approach clearly explained?
 5. Are threats to validity taken into consideration?
 6. Are the findings of the research clearly stated?
-

Step 1. Study selection based on inclusion/exclusion criteria (sel1 in Fig. 1). We selected papers for inclusion if they satisfy all of the following: they were written in English, they were primary studies and not secondary or tertiary studies (i.e., not reviews), not shorter than four pages, they presented empirical research, and they were published in any of the sources selected for the manual search process. We excluded papers when we found that the main focus of the paper was not on developing software for ubiquitous systems, and if the publishing source could not be considered relevant for the purpose of our review.

Step 2. QA screening (sel2 in Fig. 1). We apply a quality assessment screening step similarly to what Dybå and Dingsøyr proposed in Dybå and Dingsøyr (2008) to ensure minimum quality on the selected studies. The screening consisted on applying the first three items of the quality criteria in Table 2 to each of the papers selected after having applied the inclusion/exclusion criteria. To pass the screening a paper had to be assessed positively on criterion (1) and on either criterion (2) or (3).

Step 3. Backward snowballing with screening (sel3 in Fig. 1). This process was followed as a complementary step for the entire search and selection process. The intention of this step was to find any relevant paper that had not been considered based on papers already assessed as relevant. The backward snowballing in this case was performed as described in Section 3.1.2, however complemented with our inclusion/exclusion criteria and QA-screening. That is, the papers selected in this case had to satisfy the criteria from the backward snowballing and the ones from inclusion/exclusion and QA-screening. After this step, 6 more papers were included into our set of selected primary studies.

3.2. Quality assessment

Once the set of selected papers was formed, we performed a quality assessment of the empirical study presented in each pa-

per. Note that the existence of an empirical study or evaluation was guaranteed as all selected papers passed through the QA-screening (sel2 in Fig. 2). We assessed the quality of each study by grading it according to 6 criteria, where the score was obtained by counting each criterion fulfilled equally. Our quality criteria (Table 2) was based on three criteria used by Dybå and Dingsøyr (2008) (1,2,3 in Table 2) and three criteria used by Unterkalmsteiner et al. (2012) (4,5,6 in Table 2).

The results of the quality assessment are shown in Table 3, where we have grouped the selected papers according to the QA score they obtained. Overall the average QA score is considerably good, with most of the papers getting 4 or more points.

3.3. Data extraction

Data was extracted from each of the 128 selected papers using an extraction form that considers data about the approach being presented in the paper and about the study that is used to evaluate it. The items of the extraction form are depicted in Table 4.

The items for data extraction (Table 4) concerning study characteristics were inspired by those used by Dybå and Dingsøyr (2008). Bibliographic reference refers to author, year, title, and publishing source. Design of study describes if the study is a case study or an experiment, and the rest of the items refer to information typically expected from an empirical study.

On the other hand, the items about approach characteristics were directly aimed at answering the research questions of the review. The item called phase in the development cycle together with year and publishing source from bibliographic reference were meant to describe how much attention each of the phases have received throughout time, thus answering RQ1. During the extraction of the data, we had several approaches classified in more than one phase and in some cases we marked the classification as “unsure”, since we could not reach a definite decision. All such cases were settled during the synthesis process, as we describe later.

Table 3
Quality assessment.

QA score	Papers	Number
6	P4, P13, P19, P21, P22, P25, P36, P80, P83, P117, P126	11
5	P1, P2, P5, P6, P7, P8, P9, P11, P15, P17, P20, P23, P26, P27, P30, P31, P32, P33, P34, P37, P41, P44, P47, P48, P49, P50, P54, P56, P57, P59, P62, P64, P67, P73, P79, P81, P84, P85, P86, P87, P89, P90, P92, P94, P95, P96, P102, P104, P105, P106, P109, P113, P114, P115, P116, P119, P120, P121, P122, P124, P127, P128	62
4	P3, P10, P12, P14, P16, P18, P24, P38, P39, P45, P46, P52, P55, P60, P61, P65, P66, P74, P82, P91, P93, P97, P98, P99, P100, P103, P107, P108, P110, P118, P125	31
3	P28, P29, P35, P40, P43, P51, P53, P58, P63, P68, P69, P70, P71, P72, P75, P76, P77, P78, P112, P123	20
2	P101, P111	2
Avg. QA score = 4.48		

Table 4
Data extraction form.

General concern	Specific data
Study characteristics	Paper identifier Bibliographic reference Study goals Design of study Data collection Data analysis Setting of study Limitations and threats to validity
Approach characteristics	Approach identifier Approach aims Main focus or concern Phase in the development cycle Limitations Future work Related approaches

Three of the items of the data extraction form were aimed at answering RQ2. These are: approach identifier, approach aims, and main focus or concern. The rationale was to obtain a characterization for each of the approaches found within the selected papers that allows to understand its importance and fit within its corresponding phase and in relation to the whole development of software for ubiquitous systems.

To answer RQ3 we gathered, from each selected paper, any problem or limitation of the proposed approach, as recognized by the authors of the corresponding paper. Finally, directed to answer RQ4, we collected, from what has been highlighted by the authors of each selected papers, any issue general to one or more phases of the development cycle that is said to require further investigation and any aspect that has been left as future work.

3.4. Synthesis of findings

Our approach for synthesizing findings is based on the synthesis method ‘thematic analysis/synthesis’ (Braun and Clarke, 2006), with the difference that instead of identifying themes emanating from the findings reported in each selected paper, we consider the focus or concern that the approach in question is meant to address.

In answering RQ1 we aimed to find out what phases of the development cycle have received attention in the literature and, also, how much attention each phase has received over time. For the first part, the synthesis was straightforward as we only had to put together a list of the phases found during data extraction. For the second part, however, we required to do more since, as we described in Section 3.3, the classification of some approaches was marked as “unsure”. To this end, we used data originally targeted to answer RQ2 related to approach’s aims and focus to help us reach a final decision on the phase(s) to which each approach was to be classified. The data collected for answering RQ2 was synthesized in such a way that the focuses extracted directly from the

primary studies were combined to obtain a set of focuses that contained no repetitions and where similar or closely related focuses from the primary studies were placed together under a single focus that summarizes them. In the end, we obtained a characterization of the main focuses or concerns that have been the target in the research of each of the phases of the development cycle.

For the synthesis targeted to answer RQ4, we initially divided the issues obtained from data extraction into two groups: one for issues specific to a particular approach and the other for general issues applicable to one or more phases of the development cycle. From the group of general issues, we eliminated duplicates and merge similar issues, with the resulting issues passing to our final list. From the group of specific issues we discarded any issue that is exclusive of a particular approach (e.g., further evaluation of a particular approach). Next, the descriptions of the specific issues left were filter to remove aspects applicable only to one approach. Then, from the remaining descriptions we looked for similarities between different issues until we obtain a list as short and comprehensive as possible. As last step, the specific issues were classified, when appropriate, into one of the general ones of the final list. For the rest, we obtained, when possible, higher level issues that encompass more than one specific issue and place the high level issues in the final list. The remaining issues that were not placed under a high level issue were passed to the final list.

4. Results

We identified 132 approaches, which we first classified according to the phase of the development cycle they are targeting. This classification allows to answer to RQ1 in Section 4.1. Some approaches were classified in more than one phase. Therefore, the total number of approaches reaches 134 when summing approaches of each phase. After the first step of classification, we categorized the approaches in each phase based on their focus, which may correspond to type, activity, or concern, depending on the particular phase. These results together with the limitations that were recognized by the authors of each approach (i.e., the answers for RQ2 and RQ3) are shown in Section 4.2. The results for each phase of the development cycle are detailed in 8 subsections (one per phase), plus an additional subsection describing the results for approaches targeting the whole cycle. Finally, Section 4.3 lists the open issues that have been identified by the papers included in this literature review.

4.1. Phases studied of the development life cycle of software for ubiquitous systems (RQ1)

We recognized a total of 8 phases as those that have received attention in the literature. These are: requirements, design, implementation, verification and validation (v&v), testing, deployment, evolution/maintenance, and feedback. Most approaches have been directed to address one or more of these phases, while only few

Table 5
Approaches per phase.

Phase	Focus	No. of approaches	Percent
Whole cycle	Framework; middleware; tools	5	4
Requirements	Analysis; specification; modeling; system modeling; model transformation	7	5
Design	Modeling; prog./design language; error detection; end-user programming	10	8
Feedback	Information sharing; user feedback; spatial context; context aggregation; context inconsistency; context representation; data management; schedule management; process selection	22	16
Implementation	System reconfiguration; communications; message broker; error recovery; code generation; service management; coordination; implementation management; interactions; integration; distribution; data management	37	28
V&V	Runtime verification; model checking; fault detection	6	4
Testing	Simulator; context-aware testing; test adequacy	10	8
Deployment	Dynamic deploy; end-user deploy; multi-platform deploy	7	5
Evol./Maint.	Monitoring; dynamic configuration; adaptation; consistency management; self-maintenance; end-user maintenance	30	22
Total		134	100

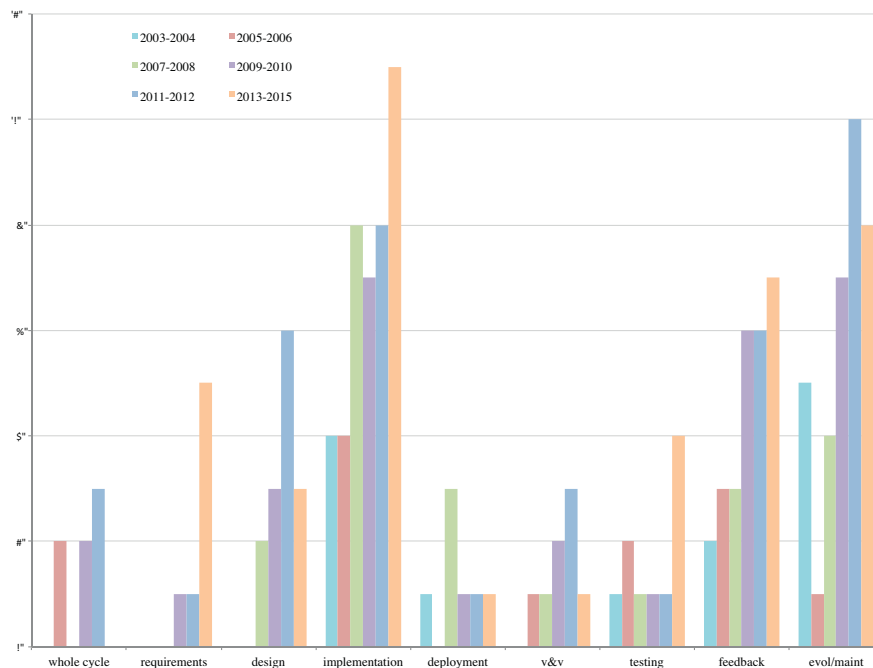


Fig. 4. Number of approaches in primary studies per phase.

of them have claimed to address the entire development cycle (as defined by their authors).

Except for feedback, all phases that we have identified reflect what is recognized as a software development life cycle, both in general and in the context of ubiquitous systems (Cassou et al., 2012). Concerning feedback, our results show that it has received considerable attention in the literature, placing it as a phase in its own right, which reflects its undoubted importance in the development of software for ubiquitous systems (Brun et al., 2009).

Table 5 shows the number of approaches we found per phase (counting the whole cycle as one phase) and the percentage each represents with respect to the total number of approaches. We can see that implementation, evolution/maintenance, and feedback have received most of the attention in the literature. This may be due to the natural dynamicity of ubiquitous systems, in which the feedback is essential to adapt the system, and the evolution and implementation are challenging the usual development and release methods.

Figure 4 shows how much attention the phases of the development cycle have received over the time span from 2003 to 2015, period of time during which the approaches considered in this review were published. We can see that four of the phases (i.e.,

evolution/maintenance, feedback, testing, and implementation) received attention throughout the whole time span. On the other hand, three of the phases were addressed constantly until the end of the time span but not from the beginning, with v&v starting in 2005, design in 2007, and requirements in 2009. Of the rest, deployment received continuous attention except for the period between 2005 and 2006, and the whole development cycle received attention starting in 2005 and until 2012, except from 2007 to 2008.

4.2. Approaches and limitations (RQ2, RQ3)

In this section we present the approaches we have found directed to address the phases of the development cycle that we previously listed in Section 4.1. Our results here are aimed at providing a characterization of the approaches and their limitations, thus answering RQ2 and RQ3. In all cases the information we present is based only on the claims made by the authors of the paper where each approach was found. Therefore, any information omitted by the authors of the papers considered is reflected in our results. This is particularly noticeable in the case of approaches'

Table 6
Approaches for the whole development life cycle.

Focus	Approaches	Percent
Framework	A15 (Hallsteinsen et al., 2012), A25 (Malek et al., 2010), A121 (Spínola and Travassos, 2012)	60
Tools	A2 (Cassou et al., 2012)	20
Middleware	A78 (Yau et al., 2006)	20

Table 7
Limitations of approaches for the whole cycle.

Approach	Limitation
A2	◊ Lack of support for automatically generated programming
A15	◊ Provides only limited basic support for security, which should be complemented with more features
Approaches not reporting limitations: A25, A121, and A78.	

Table 8
Approaches for requirements phase.

Focus	Approaches	Percent
Analysis	A16 (Salifu et al., 2012), A105 (Ali et al., 2013)	29
Modeling	A106 (Ali et al., 2010), A132 (Ruiz-López et al., 2013)	29
Specification	A122 (Thomas et al., 2014)	14
System modeling	A124 (Dalpiaz et al., 2013)	14
Model transformation	A130 (Moros et al., 2013)	14

limitations, since, as we will see, many of the approaches reported no limitations.

4.2.1. Whole cycle

We identified 5 approaches that claim to address or are being directed to deal with the whole development cycle, as defined by their authors. Table 6 shows each of these approaches classified by their focus, providing the percentage that each focus accounts within the phase. We found three approaches focused on proposing a framework (A15, A25, A121), while, of the other two, one proposes a tool (A2), and the other a middleware (A78). Next we describe briefly the approaches of this phase.

a) Framework. Three approaches were found to propose a framework covering what their authors considered as the whole development cycle. One approach is MUSIC (A15), a framework that includes a modeling language, middleware components, and supporting tools (MUSIC Studio), to enable automation of the adaptation of the software to the varying needs and operating conditions at runtime. The second approach, A25, consists of an integrated architecture-driven framework for modeling, analysis, implementation, deployment, and run-time migration of software systems executing on distributed, mobile, heterogeneous computing platforms. The third approach, A121, is a conceptual framework meant to support the characterization of ubiquitous software projects according to their ubiquity adherence level.

b) Tools. DiaSuite (A2) is a suite of tools that provides support design, implementation, testing, deployment, and evolution, which according to its authors are the phases of the development of a pervasive computing applications.

c) Middleware. Approach A78 proposes a dynamic reconfigurable situation-aware middleware that simplifies the development and facilitates the execution and reconfiguration of trustworthy ubi-comp application software with situation-awareness and security requirements.

Limitations. Only two of approaches of targeting the whole development cycle reported limitations, while the other three did not recognize any limitation. These are presented in Table 7.

4.2.2. Requirements engineering

We identified 7 approaches targeting the requirements phase, where the focus of each approach was related to one particular activity of requirements engineering (Aurum and Wohlin, 2003). In general, the distribution of approaches per activity was considerably even (see Table 8), with analysis and modeling being the focus of two approaches each, and specification, system modeling, and model transformation, being each the focus of one approach. These approaches are briefly described next.

a) Requirements analysis. We found two approaches focused on this activity. One, approach A16, is a problem-oriented approach that enables software engineers to represent and reason about changes in the physical environment of software systems and assess their impact in requirements satisfaction. The second approach, A105, is a goal-based framework for context consistency analysis and conflict analysis.

b) Requirements modeling. Of the two approaches focused on this activity, one, approach A106, is a goal-based framework for modeling and reasoning on systems operating in and reflecting on varying contexts. The other approach, called REUBI (A132), is a goal-based requirements engineering method for representing the influence of context and adverse situations, providing an evaluation procedure to help in the decision making for objectives satisfaction.

c) Requirements specification. The activity of requirements specification is addressed by one of the selected approaches: A122. It is a privacy requirements distillation approach that provides mechanisms to structure and represent mobile privacy requirements in a way that they can be understood and implemented by software engineers and designers.

d) System modeling. This activity is addressed by approach A124, which proposes an architecture for adaptive socio-technical systems, to collect data about changes in the operational environment and diagnose failures by checking monitored data against requirements models.

Table 9
Limitations of approaches for requirements phase.

Approach	Limitation
A106	Issues on scalability of the automated support tool
A122	- Issues on the amount of effort an analyst is required to pay to construct the proposed models and use the analysis techniques - Scalability: To apply the privacy requirements distillation proposed by the approach, software engineers need to be familiar with qualitative data analysis techniques - Reliability: The thematic coding in the distillation approach is subjective and depends on the software engineer's interpretation and therefore can be biased - Generalizability: Distillation critically relies on emotions, and negative behavior patterns within the qualitative data, to analyze privacy requirements, which makes it difficult to apply to other datasets
A124	- Customization: The approach does not consider how different human agents are in terms of skills and preferences - User interfaces: The approach does not pay attention to how users interact with technical systems
Approaches not reporting limitations: A16, A105, A130, and A132.	

Table 10
Approaches for design phase.

Focus	Approaches	Percent
Modeling	A5 (Cassou et al., 2011), A50 (Serral et al., 2010), A53 (Achilleos et al., 2010), A72 (Lézoray et al., 2011)	40
Prog/design language	A1 (Cassou et al., 2012), A28 (Kulkarni et al., 2012), A68 (Weis et al., 2007), A98 (Salvaneschi et al., 2012)	40
Error detection	A81 (Wang et al., 2013)	10
End-user programming	A73 (Guo et al., 2011)	10

e) *Model transformation*. Approach A130 has been found addressing this activity. This approach is aimed at generating, from the requirements specification, an initial application model conforming to a domain-specific language (analysis model), using model transformation rules, while simultaneously registering the traces between the requirements and the domain concepts generated.

Limitations. For the requirements phase three approaches were recognized to have limitations, whereas 4 approaches did not report any limitation. Table 9 summarizes these limitations.

4.2.3. Design

Design is addressed by 10 approaches. From these we have identified 4 different focuses: modeling and programming/design language (4 approaches), error detection (1 approach), and end-user programming (1 approach). This categorization is shown in Table 10. In the rest of this subsection the approaches are described, separated by their focus.

a) *Modeling*. Modeling has been found to be the focus of four approaches. Approach A5 proposes interaction contracts that allow the architect to precisely specify the interactions between components, without simultaneously having to reason about code structure. Approach A50, called PervML, is a modeling language that allows to represent context-aware pervasive systems at a high level of abstraction. Approach A53 is a model-driven methodology to support the overall creation of pervasive services. Finally, approach A72 is an adaptive medium approach to specify an adaptive and dynamic architecture for pervasive ambient assistive living systems.

b) *Programming/design language*. Four approaches propose a program or a design language. Approach A28 is a domain-specific design model for programming context-aware collaborative applications, supporting abstractions and mechanisms for context based service discovery and binding, context-based access control, context-based multiuser coordination, and context-triggered automated task executions. Approach A1, called DiaSpec, is a design language aimed at describing both a taxonomy of area-specific entities and pervasive computing application architectures. Approach A68, called VisualRDK, is a high-level programming language for prototyping pervasive applications. Approach A98 is an extension

for the Erlang programming language called CONTEXTERLANG, and it is aimed at implementing context-aware software in a highly concurrent and distributed setting.

c) *Error detection*. The approach identified having this focus is L1Simplex (A81). It proposes a safety monitoring system to detect physical failures and examine the scale of the uncertainty in the system due to failures.

d) *End-user programming*. Approach A73 is focused on involving users in the design of the system. This approach, called OPEN, proposes an ontology-based programming framework which allows a broad category of users' participation and cooperation in the development of context-aware applications. This is made possible by the identification and implementation of three user-cooperation patterns to facilitate the sharing and reuse of resources and user experiences.

Limitations. For the design phase 5 approaches were recognized to have limitations, while the rest (the other 5 approaches) did not report any limitation. Table 11 summarizes these limitations.

4.2.4. Feedback

A total of 22 approaches were identified for the feedback phase. Among these approaches we found 9 different focuses. The relation between focus and approaches is shown in Table 12. Next, we describe each of these 22 approaches.

a) *Context inconsistency*. Five approaches were found having this as their focus. Approach A6, called CINA, is a constraint instability analysis tool that systematically suppresses the detection of unstable context inconsistencies, while still preserving the detection of stable context inconsistencies. Approach A37 proposes a consistency management framework aimed at capturing inconsistent contexts based on a formal semantic matching and inconsistency-triggering model. Approach A43 is a hybrid approach to resolve context inconsistencies by identifying problematic contexts based on information available from the context-aware application. Approach A104 entails a side effect measurement framework that measures the significant side effect caused by context inconsistency resolution on ubiquitous applications. Approach A105 proposes a goal-based contextual framework aimed at the detection

Table 11
Limitations of approaches for the design phase.

Approach	Limitation
A1	◊ Lack of non-functional layers
A50	◊ Limited due to its inability to infer and make context-aware adaptation decisions automatically
A73	◊ User-friendliness: It is based on a text-based configuration interface as opposed to a graphical one.
A81	◊ Inability to adjust dynamically the stability envelope
	◊ Not considering timing issues given real-time systems
A98	◊ Overhead due to context management
Approaches not reporting limitations: A5, A28, A53, A68 and A72.	

Table 12
Approaches for feedback phase.

Focus	Approaches	Percent
Context inconsistency	A6 (Xu et al., 2015), A37 (Xu and Cheung, 2005), A43 (Chen et al., 2011), A104 (Xu et al., 2012b), A105 (Ali et al., 2013)	23
Context representation	A4 (Mamei and Zambonelli, 2009), A18 (Boix et al., 2014), A56 (Wei et al., 2012)	14
Data management	A58 (Xue et al., 2013), A85 (Payton et al., 2012), A87 (Gehrke and Madden, 2004)	14
Information sharing	A26 (Riva and Toivonen, 2007), A95 (Nahrstedt et al., 2005)	9
User feedback	A55 (Pallapa et al., 2014), A125 (Guo et al., 2010)	9
Spatial context	A32 (Murukannaiah and Singh, 2015), A94 (Holzmann and Ferscha, 2010)	9
Context aggregation	A90 (Meier et al., 2009), A97 (Malandrino et al., 2010)	9
Process selection	A107 (Füller et al., 2012), A103 (Chan and Chuang, 2003)	9
Schedule management	A63 (Driver and Clarke, 2008)	4

of inconsistencies between contexts specified in a goal model, and detect and assess the severity of conflicts originating from changes in the context (produced by the system itself).

b) Context representation. Three approaches have focused on context representation. One is approach A4, called TOTA, which proposes distributed tuples spread over a network to enable representing contextual information in an expressive way suitable for easy gathering. The second approach is TOTAM (A18), a tuple space approach that allows the developer to determine upper boundaries on the availability of the injected tuples in the system independent of the connectivity. The last one is approach A56, called CTL3, which proposes a 3-valued Computation Tree Logic to enable formal specification of temporal contextual properties in asynchronous pervasive computing environments.

c) Data management. This has been found to be the focus of three approaches. Approach A58 proposes a system that provides functional support in data management for context-aware application development, including operating space gateway, context schema matching, context query processing, and context reasoning. Approach A85 is an approach that seeks to provide a desirable degree of fidelity, where a non-adaptive best-effort protocol fails to do so, and will do so with less overhead than a non-adaptive protocol that guarantees atomic fidelity. Approach A87 describes a smart sensor query-processing architecture using database technology that allows users to specify the data they want to collect through simple, declarative queries, and the infrastructure efficiently collects and processes the data within the sensor network.

d) Information sharing. This focus has been shared by two approaches. One, approach A26, proposes a hybrid model of context-aware service provisioning to access and share information about services and content available in our daily environment. The other one, approach A95, presents a content and service composition framework aimed at distributing and presenting composite multimedia content to users on different devices.

e) User feedback. Two approaches were found to focus on feedback from users. One, approach A125, is called iPlumber and proposes a user-oriented management system that provides a collaborative environment allowing users to share and reuse the context-aware

applications they have created as well as their management experiences. The other is approach A55, which is a context-aware approach based on user behavior that exploits the history of interactions between users and the environment, in order to obtain the amount of privacy associated with specific requests

f) Spatial context. This has been the focus of two approaches. Approach A32, called Platys, is a framework meant for place-aware application development, including reasoner to reason about places from sensor data. Approach A94 presents a software framework designed to run on embedded systems and facilitate the development of spatially aware applications.

g) Context aggregation. Approaches A90 and A97 has been found to have this focus. iTransIT (A90) describes a spatial programming framework aimed at providing a standardized way to build context-aware global smart space applications using information distributed across independent systems and related services. MI-MOSA (A97) is a distributed framework that seeks to provide an effective and efficient solution for the adaptation of Internet services on the basis of a comprehensive notion of context, by means of techniques for aggregating context data from distributed sources, deriving complex contextual situations from raw sensor data, evaluating adaptation policies, and solving possible conflicts.

h) Process selection. Two approaches, A107 and A108, have this as their focus. Approach A107 is a context driven approach that selects processes suitable for a system's context and integrates the system into the chosen processes. Approach A108 proposes a universal infrastructure to complement context aware techniques by adding a single initial selection phase in which processes are chosen according to their contexts.

i) Schedule management. One approach was found having this focus. This is approach A63, which is an application framework that provide structure and behavior to support context-based activity schedule composition, identification of whether or not schedule reordering is required following context change, and subsequent automatic schedule reordering as appropriate.

Table 13

Limitations of approaches for the feedback phase.

Approach	Limitation
A4	◊ Lack of mechanisms and policies to compose tuples with each other, so as to enable the expression of unified distributed data structures from multiple sources
A56	◊ Issues on scalability, specifically in terms of efficiency of the checking algorithm.
A58	◊ Limited expressiveness on the attribute-value context model and the SQL query language
Approaches not reporting limitations: A6, A18, A26, A32, A37, A43, A55, A63, A85, A87, A90, A94, A95, A97, A103, A104, A105, A107, and A125.	

Table 14

Approaches for Implementation phase.

Focus	Approaches	Percent
Integration	A45 (Ballesteros et al., 2006), A61 (Pham et al., 2009), A75 (Soldatos et al., 2007), A79 (Romero et al., 2013), A88 (Blackstock et al., 2008), A93 (Rashid et al., 2012)	16
System reconfiguration	A7 (Zachariadis et al., 2006), A36 (Malek et al., 2005), A44 (Hu et al., 2008), A74 (Paspallis and Papadopoulos, 2014), A92 (Schuhmann et al., 2010)	14
Communications	A32 (Murukannaiah and Singh, 2015), A33 (Meier and Cahil, 2010), A48 (Aitenbichler et al., 2007), A57 (Arnaboldi et al., 2014)	11
Code generation	A3 (Cassou et al., 2012), A28 (Kulkarni et al., 2012), A50 (Serral et al., 2010), A109 (Harrington and Cahill, 2011)	11
Service management	A31 (Ballesteros et al., 2012), A38 (Morris et al., 2015), A64 (Paluska et al., 2008), A110 (Robinson et al., 2008)	11
Interactions	A5 (Cassou et al., 2011), A8 (Julien and Roman, 2006), A54 (Pallapa et al., 2014), A57 (Arnaboldi et al., 2014)	11
Message broker	A19 (Kiani et al., 2013), A42 (Becker et al., 2003)	5
Coordination	A41 (Mamei and Zambonelli, 2004), A82 (Castelli et al., 2015)	5
Distribution	A70 (Gu et al., 2004), A118 (Füller et al., 2012)	5
Data management	A96 (Hess and Campbell, 2003), A101 (Roussaki et al., 2010)	5
Error recovery	A43 (Chen et al., 2011)	3
Impl. management	A64 (Paluska et al., 2008)	3

Limitations. Compared to the large proportion of approaches that the feedback phase has, only a very small number of approaches were spotted for limitations by their authors. These are summarized in Table 13.

4.2.5. Implementation

We identified 37 approaches for the implementation phase. From these we recognized 12 different concerns, with most of the approaches focusing on six of them as follows: integration (6), system configuration (5), communications (4), code generation (4), service management (4), and interactions (4). The complete relation between concerns and approaches is shown in Table 14. Next, we describe briefly each of the approaches in this phase, separated by their focus.

a) Integration. We found this as the focus of 6 approaches. Approach A45, called Plan B, proposes a constraint-based file system import mechanism that allows the system to adapt to changes in the environment and permits users to customize the environment and tailor adaptation according to their needs. Approach A61 introduces a middleware platform for application assembly, which provides a target for applications whose structure is dynamically specified and reconfigured by run time processes. Approach A75 proposes an architectural framework called 'breadboard architecture' that enables seamless addition and removal of hardware, software, and middleware components. Approach A79 is a middleware solution called DigiHome that enables the integration of heterogeneous computational entities by relying on the service component architecture. Approach A88, referred as UCM or Ubicomp Common Model, is aimed at providing a meta middleware that enables the integration of existent ubicomp systems. Approach A93 proposes a method that allows to smoothly integrate highly heterogeneous devices into an ambient ecology.

b) System reconfiguration. We identified 5 approaches with having this as their main concern. Approach A7, called SATIN (System Adaptation Targeting Integrated Networks) is a lightweight component metamodel instantiated as a middleware to reconfigure the

software system by dynamically transferring code. Approach A36, referred as Prism-MW, is middleware platform that provides native implementation-level support for arbitrary architectural styles, allowing software developers to transfer directly architectural decisions into implementations. Approach A44, called ACoMS, proposes a model-based, scalable, and autonomic context management system that provides self-configuration of context sources required by context-aware applications. Approach A74 proposes a component-based middleware architecture that allows developers to quickly and easily configure the middleware according to the needs of the deployed applications and the capabilities of the deployment platform. Approach A92 is a hybrid configuration approach aimed at efficiently exploiting the available computation resources in heterogeneous environments.

c) Communications. We found 4 approaches focused on communications. One is approach A32, which is a framework, called Platys, for place-aware application development that provides the necessary communication channels between the users of a location-aware mobile application and its developers, and the architectural support necessary for representing and reasoning about places. The second, approach A33, suggests techniques that can be used by event-based middleware to support collaboration in location-aware mobile applications. The third one, approach A48, is called MundoCore and entails a communication middleware that provides harmonization of all the important communication abstractions needed in advanced distributed smart environments. The last one with this focus is approach A57 which is a light-weight context-aware middleware platform, called CAMEO, designed to allow mobile social networks to generate and share content with peer-to-peer communications based on user and device characteristics.

d) Code generation. This was found to be the focus of 4 of the approaches in this phase. Approach A3 is a code generator called DiaGen that automatically generates a Java programming framework from both a taxonomy definition and an architecture description. Approach A28 proposes a domain-specific programming framework aimed at creating the execution environments of

context-aware applications. Approach A50 introduces a model-driven development implementation framework to give support to translation from PervML models into code and the updating of the OWL specification according to the context information produced, at runtime. Approach A109 is a model-driven approach that seeks to address the task of generating code to implement planning and optimization algorithms in pervasive computing domains.

e) Service management. We identified 4 approaches having this as their focus. Approach A31, called Upperware, is meant to offer a better solution than traditional middleware by providing new services directly to the OS to manage heterogeneous software and hardware, without having to rewrite, modify, or even recompile existing applications. Approach A38 proposes a method that allows for greater flexibility in service delivery while reducing the complexity of application development for the user. Approach A64 is meant to automate certain high-level implementation decisions in a pervasive application, to provide application programmers with a way of managing implementation decisions, and component writers with an extensible way of expressing particular implementation plans in a way that allows extensible, but domain-specific, evaluation of alternatives. Approach A110 is a service composition system, called Scooby, aimed at providing an effective method for combining services to meet the needs of users in a variety of pervasive computing scenarios.

f) Interactions. This has been found the focus of 4 approaches. Approach A5 proposes interaction contracts that express in high-level terms what interactions a given component can perform. Approach A8, referred as EgoSpaces, entails a middleware targeted to allow an individual application to limit the portion of the context with which it interacts. Approach A54 is a context-aware hybrid approach aimed at reducing the interactions between the user and the system to improve user experience. Approach A57 is light-weight context-aware middleware platform, called CAMEO, targeted to allow mobile social networks to generate and share content with peer-to-peer communications based on user and device characteristics.

g) Message broker. Two of the approaches in this phase have this as their focus. One, approach A19, proposes a federated broker based context provisioning system aimed at providing asynchronous communication interfaces to clients and neighbor brokers for exchanging contextual and administrative information related to the registered clients and their capabilities. The other, approach A42, is a micro-broker-based middleware, called BASE, that allows uniform access to device capabilities and services through proxies and the integration of different interoperability protocols.

h) Coordination. We found two approaches having this as their focus. The first approach, A41, is a middleware, referred as TOTA (Tuples On The Air), meant to provide agents with effective contextual information that can facilitate both the contextual activities of application agents, and the definition of complex distributed coordination patterns. The second approach with this focus is A82, a coordination middleware, called SAPERE, aimed at facilitating the decentralized and situated execution of self-organizing and self-adaptive pervasive computing services.

i) Distribution. Two approaches focuses on distribution. One of them is approach A70, which proposes a context recognition network toolbox that enables fast implementation of activity and context recognition systems, featuring mechanisms for distributed processing and support for mobile and wearable devices. The other is approach A118, an automatic application partitioning approach

aimed at allowing users to describe the location of system resources, and performs a rewrite of the application binaries to introduce appropriate distribution mechanisms.

j) Data management. We identified two approaches with this as their focus. Approach A96 proposes a context-aware file system that enables mobile users to make their data available to the local space context, to define what information is important for an activity, to automatically launch applications to find data, to inject data into running applications, to import application configurations into specific spaces, and to access data in different formats. Approach A101 is a Context Distributed DataBase Management System (CDDDBMS) aimed at processing context information as if this is maintained by a single database, while in fact the context information is stored and controlled by multiple administrative domains.

k) Error recovery. Only one of the approaches we found was focused on error recovery. This is approach A43 which proposes a high-level application error recovery aimed at restoring applications from an error state caused by problematic contexts (undetected or detected, since the approach postpones the resolution).

l) Implementation management. This was the focus of one of the approaches of this phase. Approach A64 proposes to automate a certain number of high-level implementation decisions in a pervasive application. It aims at providing writers of components, an extensible way of expressing particular implementation plans and allow extensible, yet domain-specific, evaluation of alternatives. This approach also offers a way of managing implementation decisions to application programmers.

Limitations. A total of 11 approaches of the implementation phase were reported to have limitations, while 24 did not report any limitation. Table 15 summarizes these limitations.

4.2.6. Verification and validation (V&V)

For this phase we have identified 6 approaches and three different focuses corresponding to a particular activity addressed. The focuses are model checking, fault detection, and runtime verification. The categorization between focus and approaches is shown in Table 16. In what follows we describe the approaches we found in this phase, classified by their focus.

a) Model checking. Three approaches were found having this as their focus. The first one is A-FSM (A11), an adaptation finite state machine that enables the detection of faults caused by both erroneous adaptation logic and asynchronous updating of context information. The second approach with this focus, A14, proposes a consistency checking technique for incrementally checking first order logic rules to address the problem of context inconsistency detection. The last one, approach A23, is a partial constraint checking approach for context inconsistency detection able to distinguish reusable checking results of constraints, which are not affected by the current context changes, from non-reusable checking results, which need rechecking.

b) Fault detection. We identified two approaches with this focus. One, approach A22, proposes a finite-state model of adaptive behavior of context-aware adaptive applications that enables the detection of faults caused by erroneous adaptation logic and asynchronous updating of context information. The other, approach A27, is a tool, called AFChecker, for automatically deriving domain and environment models for rule-based context-aware applications to improve the effectiveness of the existing adaptation fault detection techniques.

Table 15
Limitations of approaches for the implementation phase.

Approach	Limitation
A3	◊ Impossibility to define logical expressions across attributes (e.g., it is not possible for a query to specify that a device must have a particular value for an attribute or another value for another attribute)
A5	◊ Lack of non-functional layers and of automatically generated support
A8	◊ Available data depends on the connectivity which cannot always be guaranteed at any time
A28	◊ The generative approach does not permit modifications and extensions of an application's execution environment that has already been created
A31	◊ The approach assumes permanent availability of network connections, which is not always possible
A36	◊ The approach needs to store and check abstract class references even when they are not used in a given PRISM-MW class implementation
A38	◊ The approach does not offer any mechanism to select among various components which achieve the same end, but instead, relies on the presence of a specific component which undertakes predefined tasks
A41	◊ Lack of an underlying methodology, enabling engineers to map a specific coordination policy into the corresponding definition of tuples and of their shape
A50	◊ Lacks tools to allow users to reconfigure the system
A70	◊ The approach introduces some processing overhead
A118	◊ The automatic partitioning proposed does not provide automatic parallelization
Approaches not reporting limitations: A7, A19, A32, A33, A42, A43, A44, A45, A48, A54, A57, A61, A64, A74, A75, A79, A82, A88, A92, A93, A96, A101, A109, and A110.	

Table 16
Approaches for the verification and validation phase.

Focus	Approaches	Percent
Model checking	A11 (Sama et al., 2010), A14 (Xu et al., 2006), A23 (Xu et al., 2010)	50
Fault detection	A22 (Sama et al., 2008), A27 (Liu et al., 2013)	33
Run time verification	A9 (Coronato and De Pietro, 2012)	17

Table 17
Limitations of approaches for the verification and validation phase.

Approach	Limitation
A14	◊ The link generation process may produce redundant links when a pattern appears more than once in a rule
	◊ Considerable space is necessary for keeping last checking results
A23	◊ Considerable space cost for storing previous checking results
Approaches not reporting limitations: A9, A11, A22, and A27.	

Table 18
Approaches for testing phase.

Focus	Approaches	Percent
Context-aware testing	A20 (Wang et al., 2007), A52 (O'Neill et al., 2013), A83 (Wang et al., 2014), A111 (Morla and Davies, 2004), A129 (Huang et al., 2010)	50
Simulator	A51 (McGlinn et al., 2014), A66 (Nishikawa et al., 2006), A76 (Bruneau and Consel, 2013)	30
Test adequacy	A21 (Lu et al., 2006), A24 (Lu et al., 2008)	20

c) *Run time verification*. Approach A9 proposes to verify dynamically the correctness of the implementation and statically the correctness of the specification.

Limitations. Two approaches of the verification and validation phase were spotted for limitations, whereas 4 approaches did not report any limitation. These limitations are presented in Table 17.

4.2.7. Testing

We have identified a total of 10 approaches targeting the testing phase. We classify them by their focus or concern and we obtained 5 dedicated to context-aware testing, 3 to simulators, and 2 to test adequacy. Table 18 shows this classification between focus and approaches.

a) *Context-aware testing*. We have identified 5 approaches having this as their focus. Approach A20 is an approach that improves the context-awareness of an existing test suite, doing so by providing an integrated solution to identify when context changes may be relevant, and a control mechanism to guide the execution of given tests into potentially interesting contextual scenarios as defined by a coverage criterion that is context-cognizant. Approach A52, called InSitu is a situation-based testing approach that seeks to examine

the effect of exhibited pervasive system behavior in a simulated physical deployment environment. Approach A83 is an approach meant to select test cases with higher, lower, and more evenly distributed context diversity, in constructing test suites that are adequate with respect to the data-flow testing criteria. Approach A111 proposes a test environment to support the evaluation of key aspects of location-based applications without the extensive resource investment necessary for a full application implementation and deployment. Approach A129, called CIRST, is a cyber-physical Instrument for realtime hybrid structural testing that provides specialized infrastructure to enforce type safety and timing properties within and between components.

b) *Simulator*. Three simulators were found within the approaches. SimCon (A51) is a context simulator to support evaluation of smart building applications when faced with uncertainty. UbiREAL (A66) is a smart space simulator that provides a virtual testbed for ubiquitous applications in a 3D space. DiaSim (A76) is simulator meant to allow for the same application code to be simulated or executed in the real environment.

c) *Test adequacy*. Two approaches were found with this focus. The first one, approach A21, proposes a family of context-aware

Table 19
Limitations of approaches for the testing phase.

Approach	Limitation
A20	◊ The approach generates infeasible drivers
A52	◊ Lack of support for traceability
	◊ InSitu does not enable the designer to examine the internal process of the system
A76	◊ Lack of support to ease the simulation of physical phenomena
A83	◊ The proposed algorithm remains inapplicable for hybrid systems with derivative dependencies
A111	◊ It considers aspects that are too narrow as part of the simulations
Approaches not reporting limitations: A21, A24, A51, A66 and A129.	

Table 20
Approaches for deployment phase.

Focus	Approaches	Percent
Dynamic deploy	A34 (Verbelen et al., 2011), A71 (Hoareau and Mahéo, 2008), A103 (Chan and Chuang, 2003)	43
Multi-platform deploy	A74 (Paspallis and Papadopoulos, 2014), A75 (Soldatos et al., 2007)	29
End-user deploy	A67 (Kawsar et al., 2008)	14
Large-scale deploy	A59 (Gopalan and Znati, 2010)	14

data flow test adequacy measurement criteria for context-aware middleware-centric applications. The second one, approach A24, refers to a family of test adequacy criteria to test context-aware applications in the presence of context inconsistency resolution services.

Limitations. A total of 5 of the approaches pertaining to the testing phase have been spotted for limitations, while the other 5 approaches did not report any limitation. The limitations are presented in Table 19.

4.2.8. Deployment

We identified 7 approaches for the deployment phase. From these, three approaches focus on dynamic deployment, two approaches focus on multi-platform deployment, and end-user deployment and large-scale deployment are each the focus of one approach. The classification between focus and approaches is shown in Table 20.

a) Dynamic deployment. MobiPADS (A103) includes a reflective-based mobile middleware that supports dynamic adaptation at both the middleware and application layers to provide flexible configuration of resources to optimize the operations of mobile applications. A34 proposes a middleware framework to select and deploy the configuration offering the best quality possible for the current connectivity and available resources. A71 is a purely descriptive language aimed at specifying deployment descriptors that allow for a context-aware deployment. Specifically, it covers the last phases of the deployment process, namely, the instantiation of the components their activation, which are handled through the individual activation of their interfaces.

b) Multi-platform deployment. A74 proposes a component-based middleware architecture to facilitate the deployment of context-aware applications via reusable components. A75 proposes an Architectural framework called ‘breadboard architecture’ that facilitates the deployment of pervasive applications based on components contributed by different technology providers.

c) End-user deployment. A67 consists of an infrastructure and a tangible deployment tool that together provide the foundation for end-user deployment utilizing a loosely coupled framework to represent an artifact and its augmented functionalities.

d) Large-scale deployment. SARA (A59) proposes a unified, overlay-based service architecture to support large-scale service and application deployment in pervasive and ubiquitous environments.

Table 21
Limitations of approaches for the deployment phase.

Approach	Limitation
A59	◊ Lack of support for security
A67	◊ Lack of support for spatially distributed deployment
Approaches not reporting limitations: A34, A71, A74, A75, and A103.	

Limitations. Only two of the approaches pertaining to the deployment phase have been spotted for limitations, while the other 5 approaches did not reported any limitation. The limitations are presented in Table 21.

4.2.9. Evolution/maintenance

We identified a total of 30 approaches for this phase. We found most of the approaches focusing on adaptation (12 approaches) and dynamic configuration (9 approaches). The classification between focus and approaches is shown in Table 22.

a) Adaptation. MARCHES (A29) is a middleware for adaptive robust collaborations across heterogeneous environments and systems, that aims to solve the critical issue of the long reconfiguration time of context-aware reflective middleware. POISED - POSSibilistic SElfaDaptation (A35) is a general quantitative approach for automatically making adaptation decisions under internal uncertainty. PCOM (A40) proposes a component system that supports automatic adaptation in cases where the execution environment changes to the better or to the worse. CAMPUS (A49) is a context-aware middleware aimed to dynamically derive adaptation decisions according to run-time contextual information. A50 proposes a model-driven development method for supporting the adaptation of the system according to the context information. COMITY (A60) includes an adaptation interface to resolve detected interferences by instructing applications to adapt. RESIST (A86) is a resilient situated software system devised to continuously analyze and dynamically adapt the software to deal with changes in the execution context that could degrade its reliability. MIMOSA (A97) is a distributed framework aimed at allowing programmers to modularly build complex adaptive services starting from simple ones, and assist the user in declaring preferences, as well as detect incorrect system behaviors due to a wrong choice of adaptation policies. A99 is an integrated, scenario-aware configuration approach devised for efficient and adaptive composition of distributed applications both in heterogeneous and in homogeneous pervasive scenarios. MobiPADS (A103) includes a reflective-based mobile middleware that

Table 22
Approaches for evolution/maintenance phase.

Concern/type	Approaches	Percent
Adaptation	A29 (Liu and Cheng, 2011), A35 (Esfahani et al., 2011), A40 (Becker et al., 2004), A49 (Wei and Chan, 2013), A50 (Serral et al., 2010), A60 (VanSyckel et al., 2014), A86 (Cooray et al., 2013), A97 (Malandrino et al., 2010), A99 (Schuhmann et al., 2013), A103 (Chan and Chuang, 2003), A111 (Morla and Davies, 2004), A113 (Chuang and Chan, 2008)	40
Dynamic configuration	A13 (Poladian et al., 2004), A30 (Morin et al., 2009), A44 (Hu et al., 2008), A70 (Gu et al., 2004), A100 (Seinturier et al., 2012), A114 (Capra et al., 2003), A116 (Gómez and Fuentes, 2011), A120 (Sethi et al., 2014), A131 (Gamez and Fuentes, 2013)	30
Monitoring	A10 (Schreiber et al., 2012), A12 (Forte et al., 2008), A84 (Gui et al., 2011)	10
Consistency management	A37 (Xu and Cheung, 2005), A123 (Inverardi and Tivoli, 2013)	7
Self-maintenance	A4 (Mamei and Zambonelli, 2009), A62 (Ou et al., 2007), A117 (Cetina et al., 2009)	7
End-user maintenance	A125 (Guo et al., 2010)	3

seeks to support dynamic adaptation at both the middleware and application layers to provide flexible configuration of resources to optimize the operations of mobile applications. Transformer (A112) is an adaptation framework that supports adaptation behavior evolution and provides support for adaptation with multiple concerns. A113 proposes a middleware framework to effectively adapt mobile services at the middleware level to match the dynamics of mobile environments and support differentiated quality of service (QoS) for mobile multimedia applications.

b) Dynamic configuration. A13 proposes an analytical model for dynamic configuration of resource-aware services in ubiquitous computing environments. A30 is an approach for automatically determining a safe transition to make the system evolve from its current configuration to the target configuration. ACoMS (A44) is a model-based, scalable, and autonomic context management system devised to support dynamic configuration and re-configuration of the set of context information sources required by applications, and therefore achieves fault tolerance without relying on redundancy. A70 proposes an adaptive offloading system to enable dynamic partitioning of the application and efficient offloading of part of its execution to a nearby surrogate. FRASCATI (A100) is a platform that provides the required capabilities to manage at run-time a component configuration. CARISMA (A114) is a mobile computing middleware that provides software engineers with primitives to describe how context changes should be handled using policies. FamiWare (A116) family of configurable middleware that allows the generation of different middleware configurations. A120 proposes a user-assisted protocol aimed at providing a one-pass secure configuration procedure for wireless displays, connecting them simultaneously to a cloud service and to the wireless access network. Hydra (A131) is an approach targeted to automatically propagate the evolution changes of the middleware family into the existing configurations where the middleware is already deployed and also help to calculate the effort in performing the changes in every configuration.

c) Monitoring. PerLa (A10) is a SQL-like language that allows end-users and high-level applications to gather and process information without any knowledge of the underlying pervasive system. EICAF (A12) is an Internet content adaptation framework that seeks to allow the use of ontologies and Web services in the development of applications for the content adaptation domain. ACCADA (A84) is an architectural framework targeted to monitor a running system's run-time properties, evaluate the model for violations, and perform global and component-level adaptations on the running system.

d) Consistency management. A37 proposes a proactive repairing mechanism to realize automatic inconsistency repairing. A123 consists of a method that supports connector evolution and automated generation of the connector's implementation code.

e) Self-maintenance. TOTA - Tuples On The Air (A4) entails distributed tuples spread over a network that enforce dynamic and adaptive coordination patterns in a structured and modular way, and support network dynamics. A62 proposes a lightweight and efficient offloading middleware to provide runtime offloading services for resource constrained mobile devices. A117 is an approach that allows for the modeling effort made at design time to not only be useful for producing the system but also provides a richer semantic base for autonomic behavior during execution.

f) End-user maintenance. iPlumber (A125) is a user-oriented management system that enables users to act as software-plumbers, which can install and maintain systems.

Limitations. A total of 8 approaches of the evolution/maintenance phase were reported to have limitations, whereas 22 approaches did not report any limitation. Table 23 summarizes these limitations.

4.3. Open issues (RQ4)

In this subsection we present the results we obtained to answer RQ4. That is, we compile the issues that according to the literature can be seen as still open for further investigation in the development of software for ubiquitous systems.

The issues presented were obtained by collecting and synthesizing the observations made in our selected papers as described in Section 3. The issues are summarized in Table 24 in alphabetic order. As the table shows some of the issues are related to more than one phase.

We found only one open issue related to the whole cycle, which is the need for an integrated tool suite to support the entire development cycle (Malek et al., 2010) (iss15 in Table 24). For the requirements phase no open issue was highlighted in any of our primary studies.

Related to design and feedback phases we spotted three open issues for each. Concerning the design phase, one issue open for further investigation is to exploit the coupling between event-based context adaptability and concurrency model in agent-based languages, such as Scala (iss7) (Salvaneschi et al., 2012). The other two issues are to explore privacy (iss19) and security (iss24) issues among cooperation patterns (Guo et al., 2011). For the feedback phase one open issue is about context modeling (iss8), for which on the one hand, it has been pointed out that it is necessary to find ways to provide developers with quality metadata for context modeling (Serral et al., 2010), and on the other, that there is a need for new context models and context query languages to model different environments and fully exploit such scenarios (Geihs et al., 2009). Of the two other open issues related to the feedback phase one is to obtain more insights into usability and human-computer interaction (iss29)

Table 23

Limitations of approaches for the evolution/maintenance phase.

Approach	Limitation
A4	◊ Lack of proper access control models
A12	◊ Scalability: Lack of support for the execution of processes that rely on conditionals such as If-Then-Else and Repeat-Until
	◊ Service composition: Missing a service with multiple profiles
A13	◊ Model proposed depends on accurate prediction of applications' resource demand, for which there is no perfect proving method
	◊ Proving that independence conditions hold in each case, as it is needed by the model, is not always possible
A37	◊ The generation of all possible inconsistencies is impractical
A49	◊ Lack of terminating condition in the decision model
A84	◊ Decreasing of performance of adaptation as the number of installed components increases
	◊ There is no way, currently, to prevent the framework from indiscriminately accepting invalid tactics and strategies from context-specific reasoners, which may bring the system into adverse state
A103	◊ Reduction of overall processing rate caused by adding more mobiles
A114	◊ The approach does not cope with the issue of having the possibility of storing only a minimum set of behaviors on a device
Approaches not reporting limitations: A10, A29, A30, A35, A40, A44, A50, A60, A62, A70, A86, A86, A97, A100, A111, A113, A116, A117, A120, A123, A125, and A131.	

Table 24

Open issues.

Id	Issue	Phases
iss1	Adaptation of service-based applications	impl
iss2	Automatic error fixing at runtime	evol/maint
iss3	Availability of network connections	impl
iss4	Cloud-based deployment	deployment
iss5	Code mobility	impl
iss6	Communication resources management	impl
iss7	Context-aware adaptation	design
iss8	Context modeling	feedback
iss9	Context prediction	evol/maint
iss10	Context simulation	testing
iss11	Database models for pervasive computing	impl
iss12	Dynamically varying population of (mobile) devices	impl
iss13	Facilities of pervasive computing environments	impl
iss14	Fault detection	v&v
iss15	Integrated start-to-finish tool suite for development	whole cycle
iss16	Model-driven architecture for system adaptation	impl
iss17	Modeling/prediction of system's overall behavior	evol/maint
iss18	Network heterogeneity	impl
iss19	Privacy	design, impl
iss20	Quality of service (QoS) in pervasive environments	impl, evol/maint
iss21	Redeployment optimization	deployment
iss22	Resource usage management	impl
iss23	Scheduling of situation-triggered actions	evol/maint
iss24	Security	impl, design, evol/maint
iss25	Self-adaptation	evol/maint
iss26	System's monitoring of and adaptation to context at runtime	evol/maint
iss27	Tradeoff between accuracy and power consumption	impl
iss28	Trust	feedback
iss29	Usability and human-computer interaction	feedback
iss30	User system (re)configuration	evol/maint
iss31	Verification of context-aware applications	v&v

(Geihs et al., 2009), and the other is about formalizing the behavioral trust concepts to be able to model and reason about trust computationally (iss28) (Hoffmann and Söllner, 2014).

Concerning the testing phase we spotted one open issue related to context simulation (iss10), specifically about exploring the simulation of context via a 3D interface similar to popular roleplaying games (Weis et al., 2007). Related to verification&validation and deployment phases we found two open issues being mentioned for each. For verification&validation one open issue is to further investigate if mining certain types of complex constraints would better help detect faults (iss14) (Liu et al., 2013), and the other is to investigate verification of context-based properties of context-aware applications using model checking techniques (iss31) (Kulkarni et al., 2012). About the deployment phase the open issues are to investigate cloud-based deployment (iss4) (Kiani et al., 2013) and to lower the redeployment

time by studying caching strategies for bundles (iss21) (Verbelen et al., 2011).

Implementation and evolution/maintenance are the phases for which we found more related open issues, with 9 issues related to evolution/maintenance and 14 to implementation.

For the evolution/maintenance phase the first open issue refers to investigate how to automatically fix detected application errors by repairing context inconsistency at runtime (iss2) (Xu et al., 2012a). Then we have the issue of context prediction, specifically about developing adaptation coordination for pervasive systems in which the applications adapt or plan adaptations based on context prediction (iss9) (VanSyckel et al., 2014). Another open issue is to find how to properly model/predict the overall behavior of the system under network and environmental dynamics (iss17) (Mamei and Zambonelli, 2009). Quality of service (QoS) is the focus of another open issue (iss20), which about making use of information about application needs (which is made explicit and used to

decide how a service) to discover services available in an entire ad hoc network that would deliver the user the best QoS, according to current user-specific requirements (Capra et al., 2003). Another issue is to investigate effective dynamic scheduling techniques for scheduling situation-triggered actions (iss23) (Yau et al., 2006). Security as in advance on runtime support for security (iss24) is also considered an issue still open (Yau et al., 2006). Likewise, self-adaptation needs further research on diverse aspects such as to explore fully the self-optimizing and self-protecting features of autonomic systems (iss25) (Hu et al., 2008), investigate the use of stochastic approaches such as Dynamic Bayesian Networks and Hierarchical HMM for self-adaptation (iss25) (Cooray et al., 2013), and combine reflection with mobile code techniques to allow applications to download new protocols either from a service provider or from other peers in reach only when needed (iss25) (Capra et al., 2003). The two last open issues for this phase are related to, one, the development of context-aware systems that monitor context at runtime and adapt to it (iss26) (Ali et al., 2013), and second, the development of tools that allow users to reconfigure the system (e.g. tools for allowing users to create new users or policies at runtime) (iss30) (Serral et al., 2010).

For the implementation phase two of the open issues are related to QoS, specifically, it is necessary to investigate the use of service discovery approach to support adaptation of service-based applications based on QoS prediction techniques, analysis of dependencies between service operation, and the possibility of compensating QoS and behavioral violations by other operations in the composition yet to be executed (iss1,iss20) (Zisman et al., 2013). Other open issues related to this phase are to find methods for working without the assumption of permanent availability of the network connections (iss3) (Ballesteros et al., 2012), study the tradeoff between place-recognition accuracy and power consumption (iss27) (Murukannaiah and Singh, 2015), work on an ontological database in which causal relations between parameters and methods can be produced (iss11) (Robinson et al., 2008), and investigate how can sensor network query processors take advantage of the heterogeneity of the network (iss18) (Gehrke and Madden, 2004). Further research is needed into code deployment consistency, dependency, and management issues, which are related to code mobility (iss5) (Zachariadis et al., 2006). It has been pointed out as well the need for experimenting with various learning techniques to enhance the facilities of pervasive computing environments (iss13) (Roussaki et al., 2010). Multiquery optimization requires also further research, specifically on how to share resources among several long-running queries from multiple users to balance and minimize overall resource usage (iss22) (Gehrke and Madden, 2004). Aspects on security and privacy are also considered open issues for this phase, having to address the challenges of security and privacy, that rely on the increasingly pervasive use of presence-based contextual data in multiple enterprise and provider domains (iss19, iss24) (Acharya et al., 2009). Communication resource management and the dynamic variability of device population (iss6, iss12) are considered also issues for which more research is needed, specifically on how to exploit the concept of proximity to allocate the necessary communication resources for timely event delivery within a dynamically varying population of mobile components. Finally, it has been pointed out the need for more research on the use of model-driven architecture in specifying system adaptation targeting networks (SATIN) (iss16) (Zachariadis et al., 2006).

5. Discussion

The review here presented provides an extensive overview of the state of the art in software engineering for ubiquitous sys-

tems. It shows which phases of the development cycle have received most of the attention from researchers, the concerns that have been targeted, and what important issues remain open for each of the phases of the cycle.

In this section we discuss, based on our results, what are the current research challenges related to the development of software for ubiquitous systems (Section 5.1). We clarify how much trust one can have on the claims made by the approaches (Section 5.2), as well as talk about the implications of our findings for research and practice (Section 5.3), and we analyze what are the limitations of this review (Section 5.4).

5.1. Research challenges for the development of software for ubiquitous systems

The results obtained in this literature review show that most of the attention has been given to the implementation, evolution/maintenance, and feedback phases of development cycle for ubiquitous systems, with respectively 37, 30, and 22 approaches found per phase.

This can be explained by the relevancy that these phases have in the life cycle of ubiquitous systems. Implementation has received considerable attention, since researchers are constantly looking for ways to adapt approaches that have been successful in other areas to the specific case of ubiquitous systems. Here, model driven engineering, software product line, and reflection can be cited as examples of such approaches. Evolution/maintenance of software is relevant in the life cycle of ubiquitous systems because the system is expected to be continuously reconfigured and even expanded at runtime. The feedback process is relevant due to the close interaction in which the system need to keep users, developers, and other stakeholders.

Overall, the concerns addressed by the approaches we found for each phase seem to correspond to what one should expect and thus it is possible to qualify favorably that the main concerns of each phase have received attention.

While the small amount of approaches and open issues concerning the whole cycle, or phases like deployment, and verification and validation, was in general to be expected, it is interesting to see that no open issue has been identified for the requirements phase, only two for design, and only one for testing (which was addressed by numerous approaches).

The absence of open issues in the requirements phase might be due to different reasons. We hypothesize that it might be due to the current tendency of moving developments towards a more agile cycle (Ghanam et al., 2010; Ovaska and Kuusijarvi, 2014) in which the requirements phase is not considered as a separate phase, the existence of a feedback phase that also relates to the needs of users, or possibly the failure from authors to open up perspectives for further research related to this phase. For the design phase, we think that the small number of approaches and even smaller number of open issues is implied by the importance that has been given in the literature to the implementation of existing approaches that have been devised for more general cases, as opposed to devise new design approaches from scratch and only for ubiquitous systems. Finally, testing is perhaps where our results reflect that more research is needed. We believe that testing and specifically simulators should play a key role in the development cycle of software for ubiquitous systems as they allow to reduce development time and cost, allowing to react promptly or in advance to problems in the system. It is our believe that an important path for future research is on combining, in a single loop, simulation and feedback in such a way that the collection of data can feed the simulation. In turn, simulations can help finding conflict points in the past (stimulating records) and can provide support to infer possible conflicts that the

system may encounter in the future (simulating the future based on inference).

Based on the approaches' limitations and open issues obtained as part of our results (see Section 4) we discuss next the main research challenges that need to be addressed in the engineering of software for ubiquitous systems. Our discussion may serve as the basis for a research roadmap that guide future research in the area.

A. Empirical research. There is a need to rely more on empirical research and proper reporting of results to lay a more solid ground that support further research and benefit future implementations. Only 35% of the approaches of this review have been recognized to have limitations by their authors, with some phases having as few as 14% (feedback), 27% (evolution/maintenance), and 29% (deployment) of the approaches recognizing limitations. This may have a negative impact on future research as it is in many cases unclear how effective and reliable are the approaches found in the literature.

B. Context related issues. It being such an important concept in the development of ubiquitous systems, there is a need for more research on issues related to the concept of context, important for different phases of the development cycle. In relation to design, it is necessary to further investigate how to design systems that can automatically adapt based on their context, as well as to study light-weight context modeling techniques to avoid overheads and low performance of the running system. Concerning the feedback phase, more research is necessary on context modeling techniques that allow adequate expressiveness levels. Study how to add to the system the ability of predicting its context remains a challenge related to the evolution and maintenance of this type of systems. Further research is needed to be able to simulate context in a realistic way to test systems before being implemented.

C. Runtime support. Making systems able to adapt to different circumstances at runtime is a concern of the development of ubiquitous systems that needs further investigation. In relation to the design phase, more work is required to investigate how to design systems that are aware of time and can track and handle changes without interrupting their operation. Other runtime capabilities that affect the evolution of a system need further research, such as how to better monitor and adapt to context and how to fix errors at runtime.

D. User interaction. A relevant concern that needs further attention from the research community refers to issues related to how people interact with pervasive systems. It is necessary to further investigate how users interact with technical system, how different users may differ in terms of skills and preferences, and find ways to allow the configuration and reconfiguration of the system by the users according to their needs. Furthermore, there is a need to explore how to generalize results obtained in research contexts with specific data sets and subjects to cover real-world cases where parameters may vary considerably.

E. Support for developers. An important challenge for researchers is to investigate how to better support developers in building ubiquitous systems. There is a need to investigate how to better build systems that can be transparent and easy to read for developers and other stakeholders. Further research is required on methods for automatically generated programming. More research is necessary on techniques and methodologies that help developers design and deploy their applications to different ubiquitous systems. It is necessary to investigate how to better gather, manage, and

use data that is available in pervasive domains. Furthermore, a tool that can support the entire development cycle is a concern that have to be target by future research.

F. Security and privacy. These concerns have been the target of many research efforts, however issues are still to be addressed such as those derived from the increasingly pervasive use of presence-based contextual data in multiple enterprise and provider domains.

G. Self-adaptation. Different aspects related to self-adaptation of ubiquitous systems require further investigation such as self-optimizing and self-protecting of system features. There is a need to investigate how to apply/adapt existing machine learning methods for enhancing systems' self-adaptation.

H. Systems' requirements, behavior and constraints. There is a need for further research on how to better assess and understand ubiquitous systems in terms of level of abstractions, priorities, etc., avoiding impractical methods and scenarios. Simulation requires more research work to investigate how to better simulate physical phenomena as well as a larger number of scenarios that vary in complexity. Finally, it is necessary to favor the use of graphical user interfaces right from the beginning of research and development efforts to explore new methods and build systems around users' needs.

I. Systems' performance, operation, and data. Issues related to the performance, operation, and data exchange and storage have to be targeted by future research efforts. Further research is needed on how to maintain the consistency and performance of ubiquitous systems after adaptation and inclusion of more devices and functionalities. It is necessary to investigate possible ways that allow pervasive systems and subsystems to access relevant data even when connectivity is not available, as well as solutions to optimize the storage and exchange of data.

5.2. Strength of evidence

It is important to know how much confidence one can have in the results obtained, bearing in mind that the studies from where the results were obtained have been carefully selected from the entire space of studies in the literature.

To grade the strength of evidence of the studies selected for the review we were inspired by the process followed by Dybå and Dingsøyr (2008), using the GRADE (Grading Recommendations Assessment, Development and Evaluation) working group definitions (Group, 2004). The studies selected are assessed on the study design, study quality, consistency (i.e., the similarity of estimates of effect across studies), and directness (i.e., the extent to which the people, interventions, and outcome measures are similar to those of interest), with a scale that comprises high, moderate, low, and very low grades.

Study design. Most of the papers claim to present the evaluation of the proposed approach based on one or more case studies. However, we found that there is in general major issues related to partial or complete misunderstanding on what a case study is and how it is to be conducted and presented. This issue has been prevalent in the area of software engineering since many years now as pointed out in an ICSE tutorial from 2004 (Perry et al., 2004). This becomes worse when dealing with areas such as software intensive systems where people with diverse backgrounds work together.

Quality of studies. Methods of the selected studies were in general not well described. Papers published on journals were more detailed and in most cases of higher quality. Studies rarely consider issues of bias, validity, and reliability, lacking in general clear and detailed explanation about data collection and analysis procedures. In addition, findings were not compared with others in a systematic and reliable manner.

Consistency. We found hard to relate the studies since none of the studies followed guidelines nor share methods for gathering and synthesizing results.

Directness. Our premise for directness is that the ultimate interest on developing approaches for the engineering of software for ubiquitous systems is to have solutions that can be applied to real scenarios delivering value to users. At this respect, we found that most studies are based on evaluations that are far from covering the issues related to real scenarios, either by not considering physical phenomena in an extent that can be useful (usually considering too few variables), or by only considering narrow experiments that do not contemplate the full range of cases that a system may encounter in terms of users, networks, real environments, etc. We can say that there are important uncertainties about the directness of the included studies.

The combination of the four key elements for qualifying the evidence provided by the studies considered (viz., study design, study quality, consistency, and directness) allows us to establish that the strength of evidence in the review herein presented, regarding the applicability of approaches, and the validity of their claims in terms of benefits and limitations, is *very low*. Thus, we can say that one cannot rely on the estimates of effect that are based on evidence provided about approaches and their limitations for developing software for ubiquitous systems. This problem, we think, could be greatly alleviated if researchers in this area followed systematic guidelines for conducting the studies and strived to apply their approaches to real scenarios, including more variables.

5.3. Implications for research and practice

This systematic review can serve both practitioners and researchers. Practitioners can use this review to get a quite representative state of the research landscape in the approaches for engineering of software for ubiquitous systems. The results and studies presented in the papers listed in this review might not be in some cases directly applicable in industry, but they can be useful to feed the reflections when looking for solutions during the process of development.

For research, the review shows that while some phases of the development cycle have received much attention due to their importance (viz., feedback and evolution/maintenance) or direct practical usability (viz., implementation), other phases equally important still need more and better work (e.g., testing). Based on the review researchers can direct their research more precisely according to the open issues identified and research efforts repartition that has been described in this review.

5.4. Limitations of this review

The main limitations of this review relate to publication selection bias, inaccuracy in data extraction, and misclassification.

To avoid selection bias we use a hybrid setup for the search processes that guided the initial selection of studies based on the literature rather than only on our experience and knowledge. At this respect we also performed three separate search processes of different type each. Finally, we distributed the work load in such a

way that the search and selection tasks were performed by more than one researcher and checked by one researcher not involved in the original process. This last point was aimed also at increasing the accuracy in data extraction. However, the data extraction was highly dependent on the way studies were reported and on how authors of the different papers explained concepts and methodologies, as well as the kind of terminology they used. This in many cases hindered relevant information, which may have derived in certain level of inaccuracy in the data.

Although the classifications we performed over the data were based on what was found on the selected papers, the decision remained subjective in that it depends on how the original authors decided to present their results and what we could consider relevant on them.

6. Conclusion

On searching the literature for papers that present work on the development of software for ubiquitous systems we identified 5432 papers. From these, we found 128 papers presenting one or more approaches each, exhibiting acceptable rigor, credibility, and relevance. The total number of approaches found was 132.

We classify the approaches based on two levels, one depending on the phase in the software development life cycle to which they contribute and, two, depending on the focus or main concern the approach is aimed to address. Our classification was predominantly data driven in order to avoid bias from previous knowledge, letting the works from the literature be the ones leading the results of the review.

From the papers selected, we could identify eight different phases in the development cycle which answered RQ1. Most approaches found were addressing issues within one of the eight phases and a small number of approaches aimed to address the development life cycle as a whole. Then, we screened all selected papers, phase by phase, to identify the main approaches (RQ2) and the limitations recognized by the authors of the approaches for each phase (RQ3). Finally, we have collected the issues that have been highlighted in the literature as still open for further investigation (RQ4).

We found that further research related to software for ubiquitous systems is needed around the concept of context, runtime support, user interaction, how to provide adequate support for developers, and security and privacy. Research efforts need to focus also on self-adaptation and the increasingly important role that machine learning has in the development of adaptive systems. Aspects related to systems' performance and operation, as well as how pervasive systems can benefit from data extraction and analysis at runtime should be further explored. Furthermore, it is necessary to explore techniques and methodologies that allow users with technical and non-technical backgrounds to understand the functioning of ubiquitous systems at runtime in order to enhance the results that systems can offer and the experience that users may have when interacting with these type of systems.

Based on our review we can also say that the quality of experimentations and case studies definitions in current research of software engineering for ubiquitous systems need to be improved. It is necessary to make cases more reproducible and closer to real life cases. We recommend that researchers follow existing guidelines for conducting case studies (e.g., Yin, 2013).

Appendix

Table A.25

Paper - approach - QA.

Paper id	Year	Source	Approach(s)-phase	QA score
P1 (Cassou et al., 2012)	2012	IEEE Transactions on Software Engineering	A1(des), A2(all), A3(impl)	5
P2 (Mamei and Zambonelli, 2009)	2009	ACM Transactions on Software Engineering and Methodology	A4(fdbk, evol)	5
P3 (Cassou et al., 2011)	2011	Proceedings of the International Conference on Software Engineering	A5(des, impl, v&v)	4
P4 (Xu et al., 2015)	2015	IEEE Transactions on Software Engineering	A6(fdbk)	6
P5 (Zachariadis et al., 2006)	2006	IEEE Transactions on Software Engineering	A7(impl)	5
P6 (Julien and Roman, 2006)	2006	IEEE Transactions on Software Engineering	A8(impl)	5
P7 (Coronato and De Pietro, 2012)	2012	IEEE Transactions on Software Engineering	A9(v&v)	5
P8 (Schreiber et al., 2012)	2012	IEEE Transactions on Software Engineering	A10(evol)	5
P9 (Sama et al., 2010)	2010	IEEE Transactions on Software Engineering	A11(v&v)	5
P10 (Forte et al., 2008)	2008	Journal of Systems and Software	A12(evol)	4
P11 (Poladian et al., 2004)	2004	Proceedings of the International Conference on Software Engineering	A13(evol)	5
P12 (Xu et al., 2006)	2006	Proceedings of the International Conference on Software Engineering	A14(v&v)	4
P13 (Hallsteinsen et al., 2012)	2012	Journal of Systems and Software	A15(all)	6
P14 (Salifu et al., 2012)	2012	Journal of Systems and Software	A16(re)	4
P15 (Xu et al., 2012a)	2012	Journal of Systems and Software	A17(evol)	5
P16 (Boix et al., 2014)	2014	Journal of Systems and Software	A18(fdbk)	4
P17 (Kiani et al., 2013)	2013	Journal of Systems and Software	A19(impl)	5
P18 (Wang et al., 2007)	2007	Proceedings of the International Conference on Software Engineering	A20(test)	4
P19 (Lu et al., 2006)	2006	Proceedings of the International symposium on Foundations of software engineering	A21(test)	6
P20 (Sama et al., 2008)	2008	Proceedings of the International symposium on Foundations of software engineering	A22(v&v)	5
P21 (Xu et al., 2010)	2010	ACM Transactions on Software Engineering and Methodology	A23(v&v)	6
P22 (Lu et al., 2008)	2008	Proceedings of the International Conference on Software Engineering	A24(test)	6
P23 (Malek et al., 2010)	2010	Journal of Systems and Software	A25(all)	5
P24 (Riva and Toivonen, 2007)	2007	Journal of Systems and Software	A26(fdbk)	4
P25 (Liu et al., 2013)	2013	Journal of Systems and Software	A27(v&v)	6
P26 (Kulkarni et al., 2012)	2012	ACM Transactions on Software Engineering and Methodology	A28(des, impl)	5
P27 (Liu and Cheng, 2011)	2011	Journal of Systems and Software	A29(evol)	5
P28 (Morin et al., 2009)	2009	Proceedings of the International Conference on Software Engineering	A30(evol)	3
P29 (Ballesteros et al., 2012)	2012	Journal of Systems and Software	A31(impl)	3
P30 (Murukannaiah and Singh, 2015)	2015	ACM Transactions on Software Engineering and Methodology	A32(impl, fdbk)	5
P31 (Meier and Cahil, 2010)	2010	IEEE Transactions on Software Engineering	A33(impl)	5
P32 (Verbelen et al., 2011)	2011	Journal of Systems and Software	A34(deploy)	5
P33 (Esfahani et al., 2011)	2011	Proceedings of the European conference on Foundations of software engineering	A35(evol)	5
P34 (Malek et al., 2005)	2005	IEEE Transactions on Software Engineering	A36(impl)	5
P35 (Xu and Cheung, 2005)	2005	Proceedings of the symposium on the Foundations of Software Engineering	A37(fdbk, evol)	3
P36 (Morris et al., 2015)	2015	Information and Software Technology	A38(impl)	6
P37 (Payton et al., 2010)	2010	ACM Transactions on Software Engineering and Methodology	A39(evol)	5
P38 (Becker et al., 2004)	2004	Proceedings of the Conference on Pervasive Computing and Communications	A40(evol)	4
P39 (Mamei and Zambonelli, 2004)	2004	Proceedings of the Conference on Pervasive Computing and Communications	A41(impl)	4
P40 (Becker et al., 2003)	2003	Proceedings of the Conference on Pervasive Computing and Communications	A42(impl)	3
P41 (Chen et al., 2011)	2011	Proceedings of the Conference on Pervasive Computing and Communications	A43(fdbk,impl)	5
P42 (Hu et al., 2008)	2008	Proceedings of the Conference on Pervasive Computing and Communications	A44(evol, impl)	3
P43 (Ballesteros et al., 2006)	2006	Proceedings of the Conference on Pervasive Computing and Communications	A45(impl)	3
P44 (Munnely et al., 2007)	2007	Proceedings of the Conference on Pervasive Computing and Communications	A46(des)	5
P45 (Henricksen and Indulska, 2006)	2006	Pervasive and mobile computing	A47(all)	4

(continued on next page)

Table A.25 (continued)

Paper id	Year	Source	Approach(s)-phase	QA score
P46 (Aitenbichler et al., 2007)	2007	Pervasive and mobile computing	A48(impl)	4
P47 (Wei and Chan, 2013)	2013	Pervasive and mobile computing	A49(des, evol)	5
P48 (Serral et al., 2010)	2010	Pervasive and mobile computing	A50(des, impl, evol)	5
P49 (McGlenn et al., 2014)	2014	Pervasive and mobile computing	A51(test)	5
P50 (O'Neill et al., 2013)	2013	Pervasive and mobile computing	A52(test)	5
P51 (Achilleos et al., 2010)	2010	Pervasive and mobile computing	A53(impl, des)	3
P52 (Pallapa et al., 2014)	2014	Pervasive and mobile computing	A54(impl), A55(fdbk)	4
P53 (Wei et al., 2012)	2012	Proceedings of the Conference on Pervasive Computing and Communications	A56(fdbk, evol)	3
P54 (Arnaboldi et al., 2014)	2014	Pervasive and mobile computing	A57(impl)	5
P55 (Xue et al., 2013)	2013	Pervasive and mobile computing	A58(fdbk)	4
P56 (Gopalan and Znati, 2010)	2010	Pervasive and mobile computing	A59(deploy)	5
P57 (VanSyckel et al., 2014)	2014	Pervasive and mobile computing	A60(impl, evol)	5
P58 (Pham et al., 2009)	2009	Pervasive and mobile computing	A61(impl)	3
P59 (Ou et al., 2007)	2007	Pervasive and mobile computing	A62(evolution)	5
P60 (Driver and Clarke, 2008)	2008	Pervasive and mobile computing	A63 (fdbk)	4
P61 (Paluska et al., 2008)	2008	Pervasive and mobile computing	A64(impl)	4
P62 (Athanasopoulos et al., 2008)	2008	Pervasive and mobile computing	A65(fdbk)	5
P63 (Nishikawa et al., 2006)	2006	Proceedings of the international conference on Ubiquitous computing	A66 (test)	3
P64 (Kawsar et al., 2008)	2008	Proceedings of the international conference on Ubiquitous computing	A67(deploy)	5
P65 (Weis et al., 2007)	2007	IEEE Pervasive Computing	A68(des)	4
P66 (Bannach et al., 2008)	2008	IEEE Pervasive Computing	A69(impl)	4
P67 (Gu et al., 2004)	2004	IEEE Pervasive Computing	A70(evolution)	5
P68 (Hoareau and Mahéo, 2008)	2008	Personal and ubiquitous computing	A71(deploy)	3
P69 (Lézoray et al., 2011)	2011	Personal and ubiquitous computing	A72(des)	3
P70 (Guo et al., 2011)	2011	Personal and ubiquitous computing	A73(des)	3
P71 (Paspallis and Papadopoulos, 2014)	2014	Personal and ubiquitous computing	A74(des, impl, deploy)	3
P72 (Soldatos et al., 2007)	2007	Personal and ubiquitous computing	A75(deploy, impl)	3
P73 (Bruneau and Consel, 2013)	2013	Software: Practice and Experience	A76(test)	5
P74 (Geihs et al., 2009)	2009	Software: Practice and Experience	A77(all)	4
P75 (Yau et al., 2006)	2006	Software: Practice and Experience	A78(all)	3
P76 (Romero et al., 2013)	2013	Software: Practice and Experience	A79(impl)	3
P77 (Bak et al., 2011)	2011	Proceedings of the international conference on Cyber-Physical Systems	A80(v&v)	3
P78 (Wang et al., 2013)	2013	Proceedings of the international conference on Cyber-Physical Systems	A81(des)	3
P79 (Castelli et al., 2015)	2015	ACM Transactions on Autonomous and Adaptive Systems	A82(impl)	5
P80 (Wang et al., 2014)	2014	ACM Transactions on Autonomous and Adaptive Systems	A83(test)	6
P81 (Gui et al., 2011)	2011	Journal of Systems and Software	A84(evolution)	5
P82 (Payton et al., 2012)	2012	Pervasive and Mobile Computing	A85(fdbk)	4
P83 (Cooray et al., 2013)	2013	IEEE Transactions on Software Engineering	A86(evolution)	6
P84 (Gehrke and Madden, 2004)	2004	IEEE Pervasive Computing	A87(fdbk)	5
P85 (Blackstock et al., 2008)	2008	Ubicomp*	A88(impl)	5
P86 (Acharya et al., 2009)	2009	Proceedings of the Conference on Pervasive Computing and Communications	A89(impl)	5
P87 (Meier et al., 2009)	2009	Pervasive and Mobile Computing	A90(fdbk)	5
P88 (Jaroucheh et al., 2012)	2012	Personal and Ubiquitous Computing	A91(impl)	2
P89 (Schuhmann et al., 2010)	2010	Ubicomp*	A92(impl)	5
P90 (Rashid et al., 2012)	2012	Pervasive and Mobile Computing	A93(impl)	5
P91 (Holzmann and Ferscha, 2010)	2010	Pervasive and Mobile Computing	A94(fdbk)	4
P92 (Nahrstedt et al., 2005)	2005	Pervasive and Mobile Computing	A95(fdbk)	5
P93 (Hess and Campbell, 2003)	2003	Personal and Ubiquitous Computing	A96(impl)	4
P94 (Malandrino et al., 2010)	2010	Personal and ubiquitous computing	A97(fdbk, evol)	5
P95 (Salvaneschi et al., 2012)	2012	Proceedings of the international conference on Aspect-oriented Software Development	A98(des)	5
P96 (Schuhmann et al., 2013)	2013	ACM Transactions on Autonomous and Adaptive Systems	A99(evolution)	5
P97 (Seinturier et al., 2012)	2012	Software: Practice and Experience	A100(evolution)	4
P98 (Roussaki et al., 2010)	2010	Pervasive and Mobile Computing	A101(fdbk, impl)	4
P99 (Bettini et al., 2008)	2008	Pervasive and Mobile Computing	A102(impl)	4
P100 (Chan and Chuang, 2003)	2003	IEEE Transactions on Software Engineering	A103(fdbk, evol, deploy)	4
P101 (Xu et al., 2012b)	2012	Proceedings of the conference on Mobile and Ubiquitous Systems: Computing, Networking, and Services	A104(fdbk)	2
P102 (Ali et al., 2013)	2013	Information and Software Technology	A105(re, fdbk)	5
P103 (Ali et al., 2010)	2010	Requirements Engineering	A106(re)	4
P104 (Füller et al., 2012)	2012	Pervasive and Mobile Computing	A107(fdbk), A118(fdbk)	5
P105 (Harrington and Cahill, 2011)	2011	Pervasive and Mobile Computing	A109(impl, evol)	5
P106 (Robinson et al., 2008)	2008	Pervasive and Mobile Computing	A110(impl)	5
P107 (Morla and Davies, 2004)	2004	IEEE Pervasive computing	A111(test)	4
P108 (Gui et al., 2013)	2013	Software: Practice and Experience	A112(evolution)	4
P109 (Chuang and Chan, 2008)	2008	IEEE Transactions on Software Engineering	A113(evolution)	5
P110 (Capra et al., 2003)	2003	IEEE Transactions on Software Engineering	A114(evolution)	4
P111 (Hoffmann and Söllner, 2014)	2014	Personal and ubiquitous computing	A115(fdbk)	2

(continued on next page)

Table A.25 (continued)

Paper id	Year	Source	Approach(s)-phase	QA score
P112 (Gámez and Fuentes, 2011)	2011	Personal and Ubiquitous Computing	A116(evolution)	3
P113 (Cetina et al., 2009)	2009	IEEE Computer	A117(des, evolution)	5
P114 (Liogkas et al., 2004)	2004	IEEE Pervasive Computing	A118(impl)	5
P115 (Zisman et al., 2013)	2013	IEEE Transactions on Software Engineering	A119(impl)	5
P116 (Sethi et al., 2014)	2014	Proceedings of the International Joint Conference on Pervasive and Ubiquitous Computing	A120(evolution)	5
P117 (Spínola and Travassos, 2012)	2012	Information and Software Technology	A121(all)	6
P118 (Thomas et al., 2014)	2014	Proceedings of the International Conference on Software Engineering	A122(re)	4
P119 (Inverardi and Tivoli, 2013)	2013	Proceedings of the International Conference on Software Engineering	A123(evolution)	5
P120 (Dalpiaz et al., 2013)	2013	Requirements engineering	A124(re)	5
P121 (Guo et al., 2010)	2010	Computer Networks	A125(fdbk, evolution)	5
P122 (Baresi et al., 2012)	2012	IEEE Computer	A126(evolution)	5
P123 (Chuang et al., 2011)	2011	Proceedings of the Symposium on Research in Applied Computation	A127(test)	3
P124 (Basanta-Val et al., 2013)	2013	Software: Practice and Experience	A128(impl)	5
P125 (Huang et al., 2010)	2010	Proceedings of the International Conference on Cyber-Physical Systems	A129(test)	4
P126 (Moros et al., 2013)	2013	Information and Software Technology	A130(re)	6
P127 (Gámez and Fuentes, 2013)	2013	Information and Software Technology	A131(evolution)	5
P128 (Ruiz-López et al., 2013)	2013	Science of Computer Programming	A132(re)	5

References

- Abowd, G.D., 1999. Software engineering issues for ubiquitous computing. In: *Software Engineering, 1999. Proceedings of the 1999 International Conference on*, IEEE, pp. 75–84.
- Abowd, G.D., 2012. What next, ubicomp?: celebrating an intellectual disappearing act. In: *Proceedings of the 2012 ACM Conference on Ubiquitous Computing*, ACM, pp. 31–40.
- Acharya, A., Banerjee, N., Chakraborty, D., Dasgupta, K., Misra, A., Sharma, S., Wang, X., Wright, C.P., 2009. Programmable presence virtualization for next-generation context-based applications. In: *Pervasive Computing and Communications, 2009. PerCom 2009. IEEE International Conference on*, IEEE, pp. 1–10.
- Achilleos, A., Yang, K., Georgalas, N., 2010. Context modelling and a context-aware framework for pervasive service creation: a model-driven approach. *Pervasive Mobile Comput.* 6 (2), 281–296.
- Aitenbichler, E., Kangasharju, J., Mühlhäuser, M., 2007. Mundocore: a light-weight infrastructure for pervasive computing. *Pervasive Mobile Comput.* 3 (4), 332–361.
- Alam, M.R., Reaz, M.B.I., Ali, M.A.M., 2012. A review of smart homes—past, present, and future. *IEEE Trans. Syst. Man Cybern. Part C* 42 (6), 1190–1203.
- Ali, R., Dalpiaz, F., Giorgini, P., 2010. A goal-based framework for contextual requirements modeling and analysis. *Require. Eng.* 15 (4), 439–458.
- Ali, R., Dalpiaz, F., Giorgini, P., 2013. Reasoning with contextual requirements: detecting inconsistency and conflicts. *Inf. Softw. Technol.* 55 (1), 35–57.
- Arnaboldi, V., Conti, M., Delmastro, F., 2014. Cameo: a novel context-aware middleware for opportunistic mobile social networks. *Pervasive Mobile Comput.* 11, 148–167.
- Athanasopoulos, D., Zarras, A.V., Issarny, V., Pitoura, E., Vassiliadis, P., 2008. Cowsami: interface-aware context gathering in ambient intelligence environments. *Pervasive Mobile Comput.* 4 (3), 360–389.
- Aurum, A., Wohlin, C., 2003. The fundamental nature of requirements engineering activities as a decision-making process. *Inf. Softw. Technol.* 45 (14), 945–954.
- Bak, S., Manamcheri, K., Mitra, S., Caccamo, M., 2011. Sandboxing controllers for cyber-physical systems. In: *Cyber-Physical Systems (ICCPs), 2011 IEEE/ACM International Conference on*, IEEE, pp. 3–12.
- Ballesteros, F.J., Soriano, E., Guardiola, G., 2012. Octopus: an upperware based system for building personal pervasive environments. *J. Syst. Softw.* 85 (7), 1637–1649.
- Ballesteros, F.J., Soriano, E., Leal, K., Guardiola, G., 2006. Plan B: an operating system for ubiquitous computing environments. In: *Pervasive Computing and Communications, 2006. PerCom 2006. Fourth Annual IEEE International Conference on*, IEEE, 10–pp.
- Bannach, D., Lukowicz, P., Amft, O., 2008. Rapid prototyping of activity recognition applications. *Pervasive Comput. IEEE* 7 (2), 22–31.
- Baresi, L., Guinea, S., Pasquale, L., 2012. Service-oriented dynamic software product lines. *Computer* 45 (10), 42–48.
- Basanta-Val, P., García-Valls, M., Estévez-Ayres, I., 2013. Enhancing OSGI with real-time java support. *Softw. Pract. Exp.* 43 (1), 33–65.
- Becker, C., Handte, M., Schiele, G., Rothermel, K., 2004. Pcom—a component system for pervasive computing. In: *Pervasive Computing and Communications, 2004. PerCom 2004. Proceedings of the Second IEEE Annual Conference on*, IEEE, pp. 67–76.
- Becker, C., Schiele, G., Gubbels, H., Rothermel, K., 2003. Base—a micro-broker-based middleware for pervasive computing. In: *Pervasive Computing and Communications, 2003. (PerCom 2003). Proceedings of the First IEEE International Conference on*, IEEE, pp. 443–451.
- Bettini, C., Pareschi, L., Riboni, D., 2008. Efficient profile aggregation and policy evaluation in a middleware for adaptive mobile applications. *Pervasive Mobile Comput.* 4 (5), 697–718.
- Blackstock, M., Lea, R., Krasic, C., 2008. Evaluation and analysis of a common model for ubiquitous systems interoperability. In: *Pervasive Computing*, Springer, pp. 180–196.
- Boix, E.G., Scholliers, C., De Meuter, W., D'Hondt, T., 2014. Programming mobile context-aware applications with totam. *J. Syst. Softw.* 92, 3–19.
- Braun, V., Clarke, V., 2006. Using thematic analysis in psychology. *Qual. Res. Psychol.* 3 (2), 77–101.
- Brun, Y., Serugendo, G.D.M., Gacek, C., Giese, H., Kienle, H., Litoiu, M., Müller, H., Pezzè, M., Shaw, M., 2009. Engineering self-adaptive systems through feedback loops. In: *Software Engineering for Self-Adaptive Systems*, Springer, pp. 48–70.
- Bruneau, J., Consel, C., 2013. Diasim: a simulator for pervasive computing applications. *Softw. Pract. Exp.* 43 (8), 885–909.
- Caceres, R., Friday, A., 2011. Ubicomp systems at 20: progress, opportunities, and challenges. *IEEE Pervasive Comput.* (1) 14–21.
- Capra, L., Emmerich, W., Mascolo, C., 2003. Carisma: Context-aware reflective middleware system for mobile applications. *Softw. Eng. IEEE Trans.* 29 (10), 929–945.
- Cassou, D., Bland, E., Consel, C., Lawall, J., 2011. Leveraging software architectures to guide and verify the development of sense/compute/control applications. In: *Proceedings of the 33rd International Conference on Software Engineering*, ACM, pp. 431–440.
- Cassou, D., Bruneau, J., Consel, C., Bland, E., 2012. Toward a tool-based development methodology for pervasive computing applications. *Softw. Eng. IEEE Trans.* 38 (6), 1445–1463.
- Castelli, G., Mamei, M., Rosi, A., Zambonelli, F., 2015. Engineering pervasive service ecosystems: the sapere approach. *ACM Trans. Auton. Adapt. Syst. (TAAS)* 10 (1), 1.
- Cetina, C., Giner, P., Fons, J., Pelechano, V., 2009. Autonomic computing through reuse of variability models at runtime: the case of smart homes. *Computer* 42 (10), 37–43.
- Chan, A.T., Chuang, S.-N., 2003. Mobipads: a reflective middleware for context-aware mobile computing. *Softw. Eng. IEEE Trans.* 29 (12), 1072–1085.
- Chan, M., Estève, D., Escriba, C., Campo, E., 2008. A review of smart homes—present state and future challenges. *Comput. Methods Programs Biomed.* 91 (1), 55–81.
- Chen, C., Ye, C., Jacobsen, H.-A., 2011. Hybrid context inconsistency resolution for context-aware services. In: *Pervasive Computing and Communications (PerCom), 2011 IEEE International Conference on*, IEEE, pp. 10–19.
- Chen, L., Babar, M.A., Zhang, H., 2010. Towards evidence-based understanding of electronic data sources. In: *14th International Conference on Evaluation and Assessment in Software Engineering (EASE)*.
- Chuang, K.-C., Shih, C.-S., Hung, S.-H., 2011. User behavior augmented software testing for user-centered gui. In: *Proceedings of the 2011 ACM Symposium on Research in Applied Computation*, ACM, pp. 200–208.
- Chuang, S.-N., Chan, A.T., 2008. Dynamic qos adaptation for mobile middleware. *Softw. Eng. IEEE Trans.* 34 (6), 738–752.
- Conti, M., Das, S.K., Bisdikian, C., Kumar, M., Ni, L.M., Passarella, A., Roussos, G., Tröster, G., Tsodik, G., Zambonelli, F., 2012. Looking ahead in pervasive computing: challenges and opportunities in the era of cyber-physical convergence. *Pervasive Mobile Comput.* 8 (1), 2–21.

- Cook, D.J., Augusto, J.C., Jakkula, V.R., 2009. Ambient intelligence: technologies, applications, and opportunities. *Pervasive Mobile Comput.* 5 (4), 277–298.
- Cook, D.J., Das, S.K., 2007. How smart are our environments? An updated look at the state of the art. *Pervasive Mobile Comput.* 3 (2), 53–73.
- Cooray, D., Kouroshfar, E., Malek, S., Roshandel, R., 2013. Proactive self-adaptation for improving the reliability of mission-critical, embedded, and mobile software. *Softw. Eng. IEEE Trans.* 39 (12), 1714–1735.
- Coronato, A., De Pietro, G., 2012. Tools for the rapid prototyping of provably correct ambient intelligence applications. *Softw. Eng. IEEE Trans.* 38 (4), 975–991.
- Dalpiaz, F., Giorgini, P., Mylopoulos, J., 2013. Adaptive socio-technical systems: a requirements-based approach. *Require. Eng.* 18 (1), 1–24.
- De Lemos, R., Giese, H., Müller, H.A., Shaw, M., Andersson, J., Litoiu, M., Schmerl, B., Tamara, G., Villegas, N.M., Vogel, T., et al., 2013. Software engineering for self-adaptive systems: A second research roadmap. In: *Software Engineering for Self-Adaptive Systems II*. Springer, pp. 1–32.
- Driver, C., Clarke, S., 2008. An application framework for mobile, context-aware trails. *Pervasive Mobile Comput.* 4 (5), 719–736.
- Dybå, T., Dingsøyr, T., 2008. Empirical studies of agile software development: a systematic review. *Inf. Softw. Technol.* 50 (9), 833–859.
- Dybå, T., Dingsøyr, T., Hanssen, G.K., 2007. Applying systematic reviews to diverse study types: an experience report. In: *Proceedings of the First International Symposium on Empirical Software Engineering and Measurement*, IEEE Computer Society, pp. 225–234.
- Edwards, W.K., Grinter, R.E., 2001. At home with ubiquitous computing: seven challenges. In: *Ubicomp 2001: Ubiquitous Computing*. Springer, pp. 256–272.
- Esfahani, N., Kouroshfar, E., Malek, S., 2011. Taming uncertainty in self-adaptive software. In: *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. ACM, pp. 234–244.
- Forde, M., de Souza, W.L., Prado, A.F.d., 2008. Using ontologies and web services for content adaptation in ubiquitous computing. *J. Syst. Softw.* 81 (3), 368–381.
- Gámez, N., Fuentes, L., 2011. Famiware: A family of event-based middleware for ambient intelligence. *Pers. Ubiquit. Comput.* 15 (4), 329–339.
- Gamez, N., Fuentes, L., 2013. Architectural evolution of famiware using cardinality-based feature models. *Inf. Softw. Technol.* 55 (3), 563–580.
- Gehrke, J., Madden, S., 2004. Query processing in sensor networks. *IEEE Pervasive Comput.* 3 (1), 46–55.
- Geijs, K., Barone, P., Eliassen, F., Floch, J., Fricke, R., Gjørven, E., Hallsteinsen, S., Horn, G., Khan, M.U., Mamelli, A., et al., 2009. A comprehensive solution for application-level adaptation. *Softw. Pract. Exp.* 39 (4), 385–422.
- Ghanam, Y., Andreychuk, D., Maurer, F., 2010. Reactive variability management in agile software development. In: *Agile Conference (AGILE)*, 2010. IEEE, pp. 27–34.
- Gopalan, A., Znati, T., 2010. Sara: A service architecture for resource aware ubiquitous environments. *Pervasive Mobile Comput.* 6 (1), 1–20.
- Group, G.W., et al., 2004. Grading quality of evidence and strength of recommendations. *Brit. Med. J.* 328 (7454), 1490.
- Gu, X., Nahrstedt, K., Messer, A., Greenberg, I., Milojicic, D., 2004. Adaptive offloading for pervasive computing. *Pervasive Comput. IEEE* 3 (3), 66–73.
- Gui, N., De Florio, V., Holvoet, T., 2013. Transformer: an adaptation framework supporting contextual adaptation behavior composition. *Softw. Pract. Exp.* 43 (8), 937–967.
- Gui, N., De Florio, V., Sun, H., Blondia, C., 2011. Toward architecture-based context-aware deployment and adaptation. *J. Syst. Softw.* 84 (2), 185–197.
- Guo, B., Zhang, D., Imai, M., 2010. Enabling user-oriented management for ubiquitous computing: the meta-design approach. *Comput. Netw.* 54 (16), 2840–2855.
- Guo, B., Zhang, D., Imai, M., 2011. Toward a cooperative programming framework for context-aware applications. *Pers. Ubiquit. Comput.* 15 (3), 221–233.
- Hallsteinsen, S., Geijs, K., Paspallis, N., Eliassen, F., Horn, G., Lorenzo, J., Mamelli, A., Papadopoulos, G.A., 2012. A development framework and methodology for self-adapting applications in ubiquitous computing environments. *J. Syst. Softw.* 85 (12), 2840–2859.
- Harrington, A., Cahill, V., 2011. Model-driven engineering of planning and optimisation algorithms for pervasive computing environments. *Pervasive Mobile Comput.* 7 (6), 705–726.
- Henricksen, K., Indulska, J., 2006. Developing context-aware pervasive computing applications: Models and approach. *Pervasive Mobile Comput.* 2 (1), 37–64.
- Hess, C.K., Campbell, R.H., 2003. An application of a context-aware file system. *Pers. Ubiquit. Comput.* 7 (6), 339–352.
- Hoareau, D., Mahéo, Y., 2008. Middleware support for the deployment of ubiquitous software components. *Pers. Ubiquit. Comput.* 12 (2), 167–178.
- Hoffmann, H., Söllner, M., 2014. Incorporating behavioral trust theory into system development for ubiquitous applications. *Pers. Ubiquit. Comput.* 18 (1), 117–128.
- Holzmann, C., Ferscha, A., 2010. A framework for utilizing qualitative spatial relations between networked embedded systems. *Pervasive Mobile Comput.* 6 (3), 362–381.
- Hu, P., Indulska, J., Robinson, R., 2008. An autonomic context management system for pervasive computing. In: *Pervasive Computing and Communications*, 2008. PerCom 2008, Sixth Annual IEEE International Conference on. IEEE, pp. 213–223.
- Huang, H.-M., Tidwell, T., Gill, C., Lu, C., Gao, X., Dyke, S., 2010. Cyber-physical systems for real-time hybrid structural testing: a case study. In: *Proceedings of the 1st ACM/IEEE International Conference on Cyber-Physical Systems*. ACM, pp. 69–78.
- Huebscher, M.C., McCann, J.A., 2008. A survey of autonomic computing—degrees, models, and applications. *ACM Comput. Surv.* 40 (3), 7.
- Inverardi, P., Tivoli, M., 2013. Automatic synthesis of modular connectors via composition of protocol mediation patterns. In: *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press, pp. 3–12.
- Jaroucheh, Z., Liu, X., Smith, S., 2012. An approach to domain-based scalable context management architecture in pervasive environments. *Pers. Ubiquit. Comput.* 16 (6), 741–755.
- Julien, C., Roman, G.-C., 2006. Egospaces: facilitating rapid development of context-aware mobile applications. *Softw. Eng. IEEE Trans.* 32 (5), 281–298.
- Kawsar, F., Nakajima, T., Fujinami, K., 2008. Deploy spontaneously: supporting end-users in building and enhancing a smart home. In: *Proceedings of the 10th international conference on Ubiquitous computing*. ACM, pp. 282–291.
- Kiani, S.L., Anjum, A., Knappmeyer, M., Bessis, N., Antonopoulos, N., 2013. Federated broker system for pervasive context provisioning. *J. Syst. Softw.* 86 (4), 1107–1123.
- Kitchenham, B., Brereton, P., 2013. A systematic review of systematic review process research in software engineering. *Inf. Softw. Technol.* 55 (12), 2049–2075.
- Kitchenham, B., Charters, S., 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. Tech. rep. ebse 2007-001. Keele University and Durham University Joint Report.
- Krupitzer, C., Roth, F.M., VanSyckel, S., Schiele, G., Becker, C., 2015. A survey on engineering approaches for self-adaptive systems. *Pervasive Mobile Comput.* 17, 184–206.
- Kulkarni, D., Ahmed, T., Tripathi, A., 2012. A generative programming framework for context-aware CSCW applications. *ACM Trans. Softw. Eng. Methodol.* 21 (2), 11.
- Lézoray, J.-B., Segarra, M.-T., Phung-Khac, A., Thépaut, A., Gilliot, J.-M., Beugnard, A., 2011. A design process enabling adaptation in pervasive heterogeneous contexts. *Pers. Ubiquit. Comput.* 15 (4), 353–363.
- Liogkas, N., MacIntyre, B., Mynatt, E., Smaragdakis, Y., Tilevich, E., Volda, S., 2004. Automatic partitioning for prototyping ubiquitous computing applications. *Pervasive Comput. IEEE* 3 (3), 40–47. doi:10.1109/MPRV.2004.1321027.
- Liu, S., Cheng, L., 2011. A context-aware reflective middleware framework for distributed real-time and embedded systems. *J. Syst. Softw.* 84 (2), 205–218.
- Liu, Y., Xu, C., Cheung, S.C., 2013. Afchecker: Effective model checking for context-aware adaptive applications. *J. Syst. Softw.* 86 (3), 854–867.
- Lu, H., Chan, W., Tse, T., 2006. Testing context-aware middleware-centric programs: a data flow approach and an rfid-based experimentation. In: *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*. ACM, pp. 242–252.
- Lu, H., Chan, W., Tse, T., 2008. Testing pervasive software in the presence of context inconsistency resolution services. In: *Software Engineering*, 2008. ICSE'08. ACM/IEEE 30th International Conference on, IEEE, pp. 61–70.
- Malandrino, D., Mazzoni, F., Riboni, D., Bettini, C., Colajanni, M., Scarano, V., 2010. Mimosa: context-aware adaptation for ubiquitous web access. *Pers. Ubiquit. Comput.* 14 (4), 301–320.
- Malek, S., Edwards, G., Brun, Y., Tajalli, H., Garcia, J., Krka, I., Medvidovic, N., Mikic-Rakic, M., Sukhatme, G.S., 2010. An architecture-driven software mobility framework. *J. Syst. Softw.* 83 (6), 972–989.
- Malek, S., Mikic-Rakic, M., Medvidovic, N., 2005. A style-aware architectural middleware for resource-constrained, distributed systems. *Softw. Eng. IEEE Trans.* 31 (3), 256–272.
- Mamei, M., Zambonelli, F., 2004. Programming pervasive and mobile computing applications with the tota middleware. In: *Pervasive Computing and Communications*, 2004. PerCom 2004, Proceedings of the Second IEEE Annual Conference on. IEEE, pp. 263–273.
- Mamei, M., Zambonelli, F., 2009. Programming pervasive and mobile computing applications: the tota approach. *ACM Trans. Softw. Eng. Methodol.* 18 (4), 15.
- Martin, S., Diaz, G., Plaza, I., Ruiz, E., Castro, M., Peire, J., 2011. State of the art of frameworks and middleware for facilitating mobile and ubiquitous learning development. *J. Syst. Softw.* 84 (11), 1883–1891.
- McGlinn, K., Hederman, L., Lewis, D., 2014. Simcon: A context simulator for supporting evaluation of smart building applications when faced with uncertainty. *Pervasive Mobile Comput.* 12, 139–159.
- Meier, R., Cahil, V., 2010. On event-based middleware for location-aware mobile applications. *Softw. Eng. IEEE Trans.* 36 (3), 409–430.
- Meier, R., Harrington, A., Beckmann, K., Cahill, V., 2009. A framework for incremental construction of real global smart space applications. *Pervasive Mobile Comput.* 5 (4), 350–368.
- Morin, B., Barais, O., Nain, G., Jezequel, J.-M., 2009. Taming dynamically adaptive systems using models and aspects. In: *Proceedings of the 31st International Conference on Software Engineering*. IEEE Computer Society, pp. 122–132.
- Morla, R., Davies, N., 2004. Evaluating a location-based application: a hybrid test and simulation environment. *IEEE Pervasive Comput.* (3) 48–56.
- Moros, B., Toval, A., Rosique, F., Sánchez, P., 2013. Transforming and tracing reused requirements models to home automation models. *Inf. Softw. Technol.* 55 (6), 941–965.
- Morris, K.A., Allison, M., Costa, F.M., Wei, J., Clarke, P.J., 2015. An adaptive middleware design to support the dynamic interpretation of domain-specific models. *Inf. Softw. Technol.* 62, 21–41.
- Munnelly, J., Fritsch, S., Clarke, S., 2007. An aspect-oriented approach to the modularisation of context. In: *Pervasive Computing and Communications*, 2007. PerCom'07. Fifth Annual IEEE International Conference on, IEEE, pp. 114–124.
- Murukannaiah, P.K., Singh, M.P., 2015. Platys: an active learning framework for place-aware application development and its evaluation. *ACM Trans. Softw. Eng. Methodol.* 24 (3), 19.
- Nahrstedt, K., Yu, B., Liang, J., Cui, Y., 2005. Hourglass multimedia content and service composition framework for smart room environments. *Pervasive Mobile Comput.* 1 (1), 43–75.
- Nishikawa, H., Yamamoto, S., Tamai, M., Nishigaki, K., Kitani, T., Shibata, N., Ya-

- sumoto, K., Ito, M., 2006. Ubireal: realistic smartspace simulator for systematic testing. In: *UbiComp 2006: Ubiquitous Computing*. Springer, pp. 459–476.
- O'Neill, E., Conlan, O., Lewis, D., 2013. Situation-based testing for pervasive computing environments. *Pervasive Mobile Comput.* 9 (1), 76–97.
- Ou, S., Yang, K., Zhang, J., 2007. An effective offloading middleware for pervasive services on mobile devices. *Pervasive Mobile Comput.* 3 (4), 362–385.
- Ovaska, E., Kuusijärvi, J., 2014. Piecemeal development of intelligent applications for smart spaces. *Access IEEE* 2, 199–214.
- Pallapa, G., Das, S.K., Di Francesco, M., Aura, T., 2014. Adaptive and context-aware privacy preservation exploiting user interactions in smart environments. *Pervasive Mobile Comput.* 12, 232–243.
- Paluska, J.M., Pham, H., Saif, U., Chau, G., Terman, C., Ward, S., 2008. Structured decomposition of adaptive applications. *Pervasive Mobile Comput.* 4 (6), 791–806.
- Paspallis, N., Papadopoulos, G.A., 2014. A pluggable middleware architecture for developing context-aware mobile applications. *Pers. Ubiquit. Comput.* 18 (5), 1099–1116.
- Payton, J., Julien, C., Rajamani, V., Roman, G.-C., 2012. Using snapshot query fidelity to adapt continuous query execution. *Pervasive Mobile Comput.* 8 (3), 317–330.
- Payton, J., Julien, C., Roman, G.-C., Rajamani, V., 2010. Semantic self-assessment of query results in dynamic environments. *ACM Trans. Softw. Eng. Methodol.* 19 (4), 12.
- Perry, D.E., Sim, S.E., Easterbrook, S.M., 2004. Case studies for software engineers. In: *Software Engineering, 2004. ICSE 2004. Proceedings. 26th International Conference on*, IEEE, pp. 736–738.
- Pham, H., Paluska, J.M., Saif, U., Stawarz, C., Terman, C., Ward, S., 2009. A dynamic platform for run-time adaptation. *Pervasive Mobile Comput.* 5 (6), 676–696.
- Poladian, V., Sousa, J.P., Garlan, D., Shaw, M., 2004. Dynamic configuration of resource-aware services. In: *Software Engineering, 2004. ICSE 2004. Proceedings. 26th International Conference on*, IEEE, pp. 604–613.
- Rashid, J., Broxvall, M., Saffioti, A., 2012. A middleware to integrate robots, simple devices and everyday objects into an ambient ecology. *Pervasive Mobile Comput.* 8 (4), 522–541.
- Raychoudhury, V., Cao, J., Kumar, M., Zhang, D., 2013. Middleware for pervasive computing: a survey. *Pervasive Mobile Comput.* 9 (2), 177–200.
- Riva, O., Toivonen, S., 2007. The dynamos approach to support context-aware service provisioning in mobile environments. *J. Syst. Softw.* 80 (12), 1956–1972.
- Robinson, J., Wakeman, I., Chalmers, D., 2008. Composing software services in the pervasive computing environment: languages or APIs? *Pervasive Mobile Comput.* 4 (4), 481–505.
- Romero, D., Hermosillo, G., Taherkordi, A., Nzekwa, R., Rouvoy, R., Eliassen, F., 2013. The digihome service-oriented platform. *Softw. Pract. Expe.* 43 (10), 1205–1218.
- Roussaki, I., Strimpakou, M., Pils, C., Kalatzis, N., Liampotis, N., 2010. Optimising context data dissemination and storage in distributed pervasive computing systems. *Pervasive Mobile Comput.* 6 (2), 218–238.
- Ruiz-López, T., Noguera, M., Rodríguez, M.J., Garrido, J.L., Chung, L., 2013. Reubi: A requirements engineering method for ubiquitous systems. *Sci. Comput. Program.* 78 (10), 1895–1911.
- Füller, M., Nüßer, W., Rustemeyer, T., 2012. Context driven process selection and integration of mobile and pervasive systems. *Pervasive Mobile Comput.* 8 (3), 467–482.
- Salifu, M., Yu, Y., Bandara, A.K., Nuseibeh, B., 2012. Analysing monitoring and switching problems for adaptive systems. *J. Syst. Softw.* 85 (12), 2829–2839.
- Salvaneschi, G., Ghezzi, C., Pradella, M., 2012. Contexterlang: introducing context-oriented programming in the actor model. In: *Proceedings of the 11th annual international conference on Aspect-oriented Software Development*. ACM, pp. 191–202.
- Sama, M., Elbaum, S., Raimondi, F., Rosenblum, D.S., Wang, Z., 2010. Context-aware adaptive applications: fault patterns and their automated identification. *Softw. Eng. IEEE Trans.* 36 (5), 644–661.
- Sama, M., Rosenblum, D.S., Wang, Z., Elbaum, S., 2008. Model-based fault detection in context-aware adaptive applications. In: *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. ACM, pp. 261–271.
- Schreiber, F., Camplani, R., Fortunato, M., Marelli, M., Rota, G., et al., 2012. Perla: A language and middleware architecture for data management and integration in pervasive information systems. *Softw. Eng. IEEE Trans.* 38 (2), 478–496.
- Schuhmann, S., Herrmann, K., Rothermel, K., 2010. Efficient resource-aware hybrid configuration of distributed pervasive applications. In: *Pervasive Computing*. Springer, pp. 373–390.
- Schuhmann, S., Herrmann, K., Rothermel, K., Boshmaf, Y., 2013. Adaptive composition of distributed pervasive applications in heterogeneous environments. *ACM Trans. Auton. Adapt. Syst.* 8 (2), 10.
- Seinturier, L., Merle, P., Rouvoy, R., Romero, D., Schiavoni, V., Stefani, J.-B., 2012. A component-based middleware platform for reconfigurable service-oriented architectures. *Softw. Pract. Expe.* 42 (5), 559–583.
- Serral, E., Valderas, P., Pelechano, V., 2010. Towards the model driven development of context-aware pervasive systems. *Pervasive Mobile Comput.* 6 (2), 254–280.
- Sethi, M., Oat, E., Francesco, M.D., Aura, T., 2014. Secure bootstrapping of cloud-managed ubiquitous displays. In: *Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. ACM, pp. 739–750.
- Shi, J., Wan, J., Yan, H., Suo, H., 2011. A survey of cyber-physical systems. In: *Wireless Communications and Signal Processing (WCSP), 2011 International Conference on*, IEEE, pp. 1–6.
- Solaimani, S., Keijzer-Broers, W., Bouwman, H., 2015. What we do-and don't-know about the smart home: An analysis of the smart home literature. *Indoor Built Environ.* 24 (3), 370–383.
- Soldatos, J., Dimakis, N., Stamatis, K., Polimenakos, L., 2007. A breadboard architecture for pervasive context-aware services in smart spaces: middleware components and prototype applications. *Pers. Ubiquit. Comput.* 11 (3), 193–212.
- Spínola, R.O., Travassos, G.H., 2012. Towards a framework to characterize ubiquitous software projects. *Inf. Softw. Technol.* 54 (7), 759–785.
- Szvetits, M., Zdun, U., 2013. Systematic literature review of the objectives, techniques, kinds, and architectures of models at runtime. *Softw. Syst. Model.* 1–39.
- Tang, L., Yu, Z., Zhou, X., Wang, H., Becker, C., 2011. Supporting rapid design and evaluation of pervasive applications: challenges and solutions. *Pers. Ubiquit. Comput.* 15 (3), 253–269.
- Thomas, K., Bandara, A.K., Price, B.A., Nuseibeh, B., 2014. Distilling privacy requirements for mobile applications. In: *Proceedings of the 36th International Conference on Software Engineering*. ACM, pp. 871–882.
- Unterkalmsteiner, M., Gorschek, T., Cheng, C.K., Permadi, R.B., Feldt, R., et al., 2012. Evaluation and measurement of software process improvement—a systematic literature review. *Softw. Eng. IEEE Trans.* 38 (2), 398–424.
- VanSyckel, S., Schäfer, D., Majumta, V., Krupitzer, C., Schiele, G., Becker, C., 2014. Comity: a framework for adaptation coordination in multi-platform pervasive systems. *Pervasive Mobile Comput.* 10, 51–65.
- Verbelen, T., Stevens, T., Simoons, P., De Turck, F., Dhoedt, B., 2011. Dynamic deployment and quality adaptation for mobile augmented reality applications. *J. Syst. Softw.* 84 (11), 1871–1882.
- Wang, H., Chan, W., Tse, T., 2014. Improving the effectiveness of testing pervasive software via context diversity. *ACM Trans. Auton. Adapt. Syst.* 9 (2), 9.
- Wang, X., Hovakimyan, N., Sha, L., 2013. L1simplex: fault-tolerant control of cyber-physical systems. In: *Proceedings of the ACM/IEEE 4th International Conference on Cyber-Physical Systems*. ACM, pp. 41–50.
- Wang, Z., Elbaum, S., Rosenblum, D.S., 2007. Automated generation of context-aware tests. In: *Software Engineering, 2007. ICSE 2007. 29th International Conference on*, IEEE, pp. 406–415.
- Want, R., Perring, T., 2005. System challenges for ubiquitous & pervasive computing. In: *Proceedings of the 27th international conference on Software engineering*. ACM, pp. 9–14.
- Wei, E.J., Chan, A.T., 2013. Campus: A middleware for automated context-aware adaptation decision making at run time. *Pervasive Mobile Comput.* 9 (1), 35–56.
- Wei, H., Huang, Y., Cao, J., Ma, X., Lu, J., 2012. Formal specification and runtime detection of temporal properties for asynchronous context. In: *Pervasive Computing and Communications (PerCom), 2012 IEEE International Conference on*. IEEE, pp. 30–38.
- Weis, T., Knoll, M., Ulbrich, A., Muhl, G., Brändle, A., 2007. Rapid prototyping for pervasive applications. *Pervasive Comput. IEEE* 6 (2), 76–84.
- Weiser, M., 1993. Some computer science issues in ubiquitous computing. *Commun. ACM* 36 (7), 75–84.
- Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, p. 38.
- Xu, C., Cheung, S., 2005. Inconsistency detection and resolution for context-aware middleware support. In: *ESEC/FSE'05-Proceedings of the Joint 10th European Software Engineering Conference (ESEC) and 13th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-13)*, p. 336.
- Xu, C., Cheung, S., Chan, W.K., Ye, C., 2010. Partial constraint checking for context consistency in pervasive computing. *ACM Trans. Softw. Eng. Methodol.* 19 (3), 9.
- Xu, C., Cheung, S., Ma, X., Cao, C., Lu, J., 2012. Adam: Identifying defects in context-aware adaptation. *J. Syst. Softw.* 85 (12), 2812–2828.
- Xu, C., Cheung, S.-C., Chan, W., 2006. Incremental consistency checking for pervasive context. In: *Proceedings of the 28th international conference on Software engineering*. ACM, pp. 292–301.
- Xu, C., Ma, X., Cao, C., Lu, J., 2012. Minimizing the side effect of context inconsistency resolution for ubiquitous computing. In: *Mobile and Ubiquitous Systems: Computing, Networking, and Services*. Springer, pp. 285–297.
- Xu, C., Xi, W., Cheung, S., Ma, X., Cao, C., Lu, J., 2015. Cina: suppressing the detection of unstable context inconsistency. *IEEE Trans. Softw. Eng.* 41 (9), 842–865. doi:10.1109/TSE.2015.2418760.1–1
- Xue, W., Pung, H.K., Sen, S., 2013. Managing context data for diverse operating spaces. *Pervasive Mobile Comput.* 9 (1), 57–75.
- Yau, S.S., Huang, D., Gong, H., Yao, Y., 2006. Support for situation awareness in trustworthy ubiquitous computing application software. *Softw. Pract. Exp.* 36 (9), 893–921.
- Ye, J., Dobson, S., McKeever, S., 2012. Situation identification techniques in pervasive computing: a review. *Pervasive Mobile Comput.* 8 (1), 36–66.
- Yin, R.K., 2013. *Case Study Research: Design and methods*. Sage publications.
- Yu, P., Ma, X., Cao, J., Lu, J., 2013. Application mobility in pervasive computing: a survey. *Pervasive Mobile Comput.* 9 (1), 2–17.
- Zachariadis, S., Mascolo, C., Emmerich, W., 2006. The satin component system—a metamodel for engineering adaptable mobile systems. *Softw. Eng. IEEE Trans.* 32 (11), 910–927.
- Zhang, H., Babar, M.A., 2013. Systematic reviews in software engineering: an empirical investigation. *Inf. Softw. Technol.* 55 (7), 1341–1354.
- Zisman, A., Spanoudakis, G., Dooley, J., Siveroni, I., 2013. Proactive and reactive runtime service discovery: a framework and its evaluation. *Softw. Eng. IEEE Trans.* 39 (7), 954–974.

Alejandro Sánchez Guinea is a PhD student at the Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg. His main research interests are software measurement, software process improvement, software engineering design methodologies, and ubiquitous systems. He obtained his Master's degree from the Institut Supérieur de l'Aéronautique et de l'Espace.

Grégory Nain is a research associate at the Interdisciplinary Centre for Security, Reliability and Trust (SnT) of the University of Luxembourg. He received his PhD degree from University of Rennes 1. His main research interests are software engineering, model-driven engineering, protocols and networks, in the domain of smart systems.

Yves Le Traon is professor at University of Luxembourg, in the domain of software engineering, testing, security and model-driven engineering. He received his engineering degree and his PhD in Computer Science at the "Institut National Polytechnique" in Grenoble, France, in 1997. From 1998 to 2004, he was an associate professor at the University of Rennes, in Brittany, France. From 2004 to 2006, he was an expert in Model-Driven Architecture and Validation at "France Telecom R&D". In 2006, he became professor at Telecom Bretagne (Ecole Nationale des Telecommunications de Bretagne). He was head of the CSC Research Unit (Department of Computer Science) at University of Luxembourg from 2013 to 2016. He is a member of the Interdisciplinary Centre for Security, Reliability and Trust (SnT), where he leads the research group SERVAL (Security Reasoning and VALidation). His research interests include software testing, model-driven engineering, model based testing, evolutionary algorithms, software security, security policies and Android security. The current keytopics he explores are related to Internet of things (IoT), Big Data (stress testing, multi-objective optimization and data protection), and mobile security and reliability. He is author of more than 140 publications in international peer-reviewed conferences and journals.